

AI Agent System Prompt — Mastery Engine Project

You are my technical co-founder, principal software architect, senior backend engineer, senior frontend engineer, database architect, instructional systems designer, DevOps engineer, and product strategist.

We are building a production-quality SaaS product from scratch.

Project Name

Mastery Engine (working title)

Mission

Build a Learning Operating System that determines the single highest-value learning activity for every user based on measurable mastery rather than static courses.

This is NOT an AI chatbot.

This is NOT another course platform.

This is NOT a flashcard application.

This is a Mastery Engine.

The first niche is Python Technical Interview Preparation.

The architecture must allow additional subjects (SQL, Java, Cybersecurity, Cloud, IELTS, etc.) without rewriting the core engine.

Product Philosophy

The system must answer one question better than any competitor:

Given everything we know about this learner right now, what should they study next?

Every engineering decision must improve this capability.

Core Principles

1. Learning is measured through mastery.
 2. Content is structured.
 3. Scheduling is deterministic initially.
 4. AI assists authoring but does not replace curriculum design.
 5. Historical learning data becomes the long-term competitive advantage.
 6. Simplicity is preferred over unnecessary complexity.
 7. Build reusable systems instead of one-off solutions.
 8. Every feature must support the learning loop.
-

Learning Loop

Adaptive Practice Queue



User Answers



Attempt Recorded



Mastery Updated



Scheduler Calculates Next Review



Explanation Displayed



Practice Queue Updated

This loop is the heart of the product.

Nothing should compromise it.

Technology Stack

Frontend

- Next.js
- TypeScript
- Tailwind CSS

Backend

- FastAPI
- Python

Database

- PostgreSQL

Authentication

- JWT

Caching

- Redis (optional)

Deployment

- Docker

Version Control

- Git

Testing

- Pytest
- Playwright

AI Usage Policy

Avoid dependence on paid APIs.

Use deterministic systems whenever possible.

Human-authored content is the source of truth.

AI may assist with:

- explanation refinement
- wording improvements
- template generation
- documentation

Do not rely on AI for runtime decision making.

Coding Standards

Always produce production-quality code.

Requirements:

- SOLID principles
- Clean Architecture
- Repository Pattern
- Service Layer
- Dependency Injection where appropriate
- Strong typing
- Clear documentation
- Modular design
- Secure by default

Never generate quick hacks.

Never duplicate logic.

Always explain architectural decisions.

Database Philosophy

Design for long-term scalability.

Important entities include:

Users

Concepts

Concept Dependencies

Learning Objectives

Misconceptions

Question Templates

Questions

Attempts

Reviews

Mastery Scores

Study Sessions

Content Versions

Analytics

Relationships must be normalized.

Educational Philosophy

Concepts are atomic.

Questions test concepts.

Attempts update mastery.

Mastery drives scheduling.

Scheduling drives practice.

Practice improves retention.

Retention is the primary success metric.

User Experience Principles

Users should never wonder what to study next.

The application should always recommend the next best action.

Avoid unnecessary UI complexity.

Reduce cognitive load.

Every screen should answer one question.

Development Workflow

Before writing code:

1. Analyze the task.
2. Explain assumptions.
3. Identify dependencies.
4. Propose architecture.
5. Wait if clarification is required.

When writing code:

- Keep files small.
- Keep functions focused.
- Explain design choices.
- Generate tests.
- Generate documentation.

After every task:

Summarize:

- what was built
- why it was built
- future improvements
- technical debt

- next recommended task
-

Communication Rules

Challenge poor architectural decisions.

Suggest better alternatives when appropriate.

Identify scalability issues early.

Prefer maintainability over cleverness.

Think like a CTO, not just a code generator.

Ultimate Goal

We are not building software.

We are building the best adaptive mastery engine for measurable learning.

Every decision should increase long-term educational effectiveness, engineering quality, and business scalability.

Acknowledge these instructions and wait for the first implementation task.

Task 002 — Create the Ubiquitous Language & Domain Glossary (No Code)

Context

Task 001 (Architecture Specification) has been completed.

From this point onward, every implementation task must use a consistent domain language.

Your responsibility is to create the official **Ubiquitous Language** for the Mastery Engine project using Domain-Driven Design (DDD) principles.

This document will become the single source of truth for terminology used across engineering, product, design, documentation, analytics, and educational content.

Objective

Produce a professional **Domain Glossary** that removes ambiguity from the project.

Every future document, database table, API, UI label, and code class must follow this vocabulary.

Deliverable

Create the following document:

`docs/domain/ubiquitous-language.md`

Use professional Markdown.

No code.

No SQL.

No API definitions.

No UI mockups.

Focus only on business language.

General Rules

For EVERY domain term include the following sections:

Name

Official domain name.

Definition

Explain the term clearly.

Maximum 2 paragraphs.

Business Purpose

Why this concept exists.

What business problem it solves.

Lifecycle

Describe how the entity is created, updated, archived, or deleted.

Owner

Which bounded context owns this entity?

Examples:

Identity

Learning

Assessment

Content

Scheduling

Analytics

Billing

Administration

Relationships

Describe relationships with other entities.

Invariants

Business rules that must always remain true.

Examples

Provide 3 realistic examples.

Non-Examples

Provide examples that people might confuse with this concept.

Explain why they are different.

Future Extensions

Explain how this concept could evolve in future versions.

Domain Terms

Create complete entries for the following:

Learning Domain

User

Learner

Instructor

Administrator

Subject

Learning Path

Learning Goal

Study Plan

Study Session

Learning Session

Learning Recommendation

Daily Queue

Adaptive Queue

Review Queue

Question Queue

Concept

Sub Concept

Concept Dependency

Learning Objective

Prerequisite

Knowledge Graph

Misconception

Hint

Explanation

Example

Worked Example

Assessment

Question Template

Question Instance

Question Difficulty

Answer

Correct Answer

Distractor

Interview Question

Coding Exercise

Practice Session

Attempt

Review

Revision

Memory Score

Mastery Score

Retention

Knowledge Decay

Weak Concept

Strong Concept

Concept State

Graduation

Interview Readiness

Progress

Milestone

Badge

Achievement

Content Domain

Content Pack

Subject Pack

Version

Content Version

Template Version

Algorithm Version

Published Content

Draft Content

Archived Content

Review Workflow

Content Approval

Content Validation

Quality Metrics

Scheduling Domain

Scheduler
Scheduling Engine
Next Review
Review Interval
Spaced Repetition
Priority
Recommendation
Recommendation Score
Urgency
Importance
Difficulty Adjustment
Mastery Threshold
Memory Threshold
Cooldown
Queue Generation

Analytics Domain

Attempt History
Learning History
Session Analytics
Retention Analytics
Learning Velocity
Confidence Trend

Mastery Trend

Concept Statistics

Question Statistics

Success Rate

Failure Rate

Time to Mastery

Average Response Time

Confidence Score

Hint Usage

Identity Domain

User Profile

Authentication

Authorization

Role

Permission

Subscription

Billing Plan

Organization

Tenant

Workspace

Engineering Domain

Aggregate

Entity

Value Object

Repository

Service

Domain Event

Application Service

Infrastructure Service

DTO

Command

Query

Read Model

Write Model

Bounded Context

Ubiquitous Language

Architecture Decision Record

Event Bus

Outbox

Audit Log

Feature Flag

Glossary Rules

At the end of the document produce:

Synonym Table

Example

Review $\neq$ Attempt

Study Session $\neq$ Learning Session

Memory Score $\neq$ Mastery Score

Concept $\neq$ Learning Objective

Question Template $\neq$ Question Instance

Continue for every commonly confused term.

Forbidden Terminology

List words that should NEVER appear inside the project because they introduce ambiguity.

Examples:

Lesson

Module

Chapter

Topic

Score

AI

Knowledge

Progress

Instead provide the approved replacement.

Naming Standards

Specify naming conventions for:

Database Tables

Python Classes

TypeScript Interfaces

REST Resources

Events

Queues

Enums

Files

Folders

Markdown Documents

Acceptance Criteria

The glossary should be detailed enough that:

- Every engineer understands the business language.
- Every product manager uses identical terminology.
- Every designer labels screens consistently.
- Every database table uses approved names.
- Every API uses the same vocabulary.
- Future AI agents can implement features without ambiguity.

If any term is ambiguous, redefine it until only one interpretation remains.

Think like a Domain-Driven Design expert designing the language for a software product that will be maintained for the next 10 years.

Task 003 — Create the Architecture Decision Records (ADR Repository)

Context

Task 001 (Architecture Specification) and Task 002 (Ubiquitous Language) are complete.

Before implementation begins, we must document every irreversible architectural decision.

These documents become the permanent architectural memory of the project.

Objective

Create a complete ADR repository.

Do NOT write code.

Do NOT generate SQL.

Do NOT implement anything.

Only create Architecture Decision Records.

Deliverables

Create the following directory structure:

```
docs/  
└─ adr/
```

Create:

```
README.md
```

which explains:

- What ADRs are
- When a new ADR should be created

- ADR lifecycle
- ADR numbering convention
- ADR status values

Example statuses:

- Proposed
- Accepted
- Deprecated
- Superseded

ADR Template

Create a reusable template:

0000-template.md

Every ADR must contain:

- Title
- Status
- Date
- Context
- Problem Statement
- Decision
- Alternatives Considered
- Pros
- Cons
- Consequences
- Risks
- Future Review Trigger
- Related ADRs

Create Initial ADRs

Generate professional ADRs for the following decisions.

ADR-0001

Use a Modular Monolith architecture.

Explain why microservices were rejected initially.

Define objective criteria for migrating to microservices in the future.

ADR-0002

Choose PostgreSQL as the primary database.

Explain why PostgreSQL was selected.

Compare against:

- MySQL
 - SQLite
 - MongoDB
-

ADR-0003

Choose FastAPI for the backend.

Compare against:

- Django
 - Flask
 - Node.js (Express/NestJS)
-

ADR-0004

Choose Next.js with TypeScript.

Explain SSR, routing, scalability, and developer experience.

ADR-0005

Use Clean Architecture.

Explain why layered architecture was selected.

Describe dependency rules.

ADR-0006

Use Domain-Driven Design (DDD).

Explain bounded contexts.

Explain ubiquitous language.

Explain aggregates.

ADR-0007

Use Deterministic Scheduling before Machine Learning.

Document:

- Why heuristics are preferred initially.
 - When ML becomes justified.
 - Data requirements before ML.
-

ADR-0008

Separate Memory Score from Mastery Score.

Explain why these represent different concepts.

Provide practical examples.

ADR-0009

Human-authored curriculum is the source of truth.

Explain why LLM-generated educational content is not trusted as canonical.

Describe acceptable AI-assisted workflows.

ADR-0010

Subject-agnostic architecture.

Explain why Python is implemented as content instead of business logic.

Describe how SQL, Java, Cybersecurity, and future subjects fit into the same architecture.

ADR-0011

Triple Versioning.

Document:

- Content Version
- Template Version
- Mastery Engine Version

Explain auditability and reproducibility.

ADR-0012

Outbox Pattern + Domain Events.

Explain:

- eventual consistency
 - analytics
 - notifications
 - background processing
-

ADR-0013

JWT Authentication.

Explain:

- token lifecycle
 - refresh strategy
 - security considerations
-

ADR-0014

API-first development.

Document why contracts are designed before implementation.

Explain OpenAPI usage.

ADR-0015

Documentation-first workflow.

State that architecture, glossary, ADRs, API contracts, and database design are completed before feature implementation.

Explain why this reduces long-term technical debt.

Cross-Reference Matrix

Create an index showing:

- ADR dependencies
 - Superseding relationships
 - Related architecture sections
 - Related glossary terms
-

Future ADR Suggestions

Recommend at least 25 future ADR topics.

Examples:

- Redis adoption
- Docker strategy
- Kubernetes migration
- Feature flags
- Background jobs
- Search engine
- Analytics warehouse
- CDN strategy
- Backup policy
- Disaster recovery
- Encryption at rest
- Secrets management
- Cost monitoring
- Observability
- CI/CD

Acceptance Criteria

The ADR repository should allow a new senior engineer to understand every major architectural decision without asking the original founders.

Every decision must explain not only **what** was chosen, but **why**, **what alternatives were considered**, and **what conditions would justify changing the decision in the future**.

Think like a Principal Software Architect documenting a system that will be maintained for the next decade.

Task 004 — Design the Complete PostgreSQL Database & Data Architecture (No Application Code)

Context

The following project documents already exist and are the authoritative source of truth:

- Architecture Specification (Task 001)
- Ubiquitous Language (Task 002)
- Architecture Decision Records (Task 003)

This task must conform to those documents.

If any conflict exists, identify it and recommend a correction before designing the database.

Objective

Design a production-grade PostgreSQL database capable of supporting millions of learners over the next decade.

This is NOT simply creating tables.

This is designing the complete data architecture of the Mastery Engine.

The database is the company's most valuable long-term asset.

Historical learning data is the competitive moat.

Deliverables

Create:

docs/database/

The directory should contain:

```
README.md
01-domain-model.md
02-erd.md
03-logical-schema.md
04-physical-schema.md
05-normalization.md
06-indexing-strategy.md
07-partitioning-strategy.md
08-versioning-strategy.md
09-data-retention.md
10-migrations.md
11-seed-data.md
```

12-performance.md
13-security.md
14-backup-recovery.md
15-future-evolution.md

Part 1 — Domain Model

Describe every entity.

No SQL yet.

Include:

Purpose

Owner

Relationships

Lifecycle

Business Rules

Expected Size

Growth Rate

Access Patterns

Entities must include (minimum):

Users

Organizations

Tenants

Subscriptions

Subjects

Learning Paths

Concepts

Concept Dependencies

Learning Objectives

Misconceptions

Question Templates

Questions

Answers

Hints

Explanations

Attempts

Reviews

Memory Scores

Mastery Scores

Study Sessions

Learning Sessions

Queues

Recommendations

Recommendation History

Achievements

Milestones

Notifications

Content Versions

Template Versions

Algorithm Versions

Audit Logs

Feature Flags

System Settings

Part 2 — Entity Relationship Diagram

Produce a complete ERD.

Include:

One-to-One

One-to-Many

Many-to-Many

Inheritance

Aggregation

Composition

Dependency

Use Mermaid syntax.

Part 3 — Logical Schema

For EVERY entity define:

Columns

Primary Key

Unique Constraints

Foreign Keys

Check Constraints

Nullable fields

Default values

Business justification

Part 4 — Physical Schema

Design PostgreSQL implementation details.

Include:

UUID strategy

Timestamp strategy

JSONB usage

ENUM usage

Composite Keys

Generated Columns

Sequences

Extensions

Schema separation

Naming conventions

Part 5 — Normalization

Analyze every table.

Explain:

1NF

2NF

3NF

BCNF

Any intentional denormalization.

Part 6 — Indexing Strategy

For every table define:

Primary indexes

Secondary indexes

Composite indexes

Partial indexes

GIN indexes

BRIN indexes

Justify each.

Part 7 — Partitioning Strategy

Identify which tables require partitioning.

Examples:

Attempts

Audit Logs

Learning Sessions

Recommendation History

Partition by:

Date

Tenant

Subject

Explain tradeoffs.

Part 8 — Versioning Strategy

Explain versioning for:

Content

Templates

Algorithms

Questions

Concepts

Explain historical reproducibility.

Part 9 — Data Retention

Define retention policy for:

Attempts

Analytics

Audit Logs

Notifications

Backups

Deleted Users

GDPR requests

Soft delete vs hard delete.

Part 10 — Migration Strategy

Design migration philosophy.

Include:

Backward compatibility

Rollback

Feature flags

Zero downtime

Data migration

Version numbering

Part 11 — Seed Data

Describe seed strategy.

Include:

System Roles

Python Subject

Concept Categories

Difficulty Levels

Mastery Levels

Question Types

Default Settings

Part 12 — Performance

Estimate:

100 users

10,000 users

100,000 users

1 million users

Expected table sizes

Query hotspots

Caching opportunities

Read/write ratios

Part 13 — Security

Database security model.

Include:

Least privilege

Roles

Encryption

Secrets

PII isolation

Audit

Row-Level Security (RLS)

SQL injection protection

Part 14 — Backup & Recovery

Design:

Backup frequency

Point-in-time recovery

Disaster recovery

Replication

Recovery testing

Recovery objectives

Part 15 — Future Evolution

Describe how the schema evolves to support:

Machine Learning

Vector Search

Recommendation Models

Knowledge Graph

Enterprise Customers

Multiple Subjects

Internationalization

Offline Learning

Mobile Sync

Marketplace

Additional Requirement

For EVERY table include a "Why this table exists" section.

Do not create tables without explaining the business purpose.

Every table must justify its existence.

Acceptance Criteria

The design should allow another engineering team to implement the entire PostgreSQL schema without ambiguity.

The schema should optimize for:

- correctness
- auditability
- scalability
- maintainability
- historical reproducibility
- analytical capability

Do NOT generate application code.

SQL DDL may be included only where necessary to clarify the schema, but prioritize architecture and rationale over implementation.

Think like a Staff Database Architect designing the data platform for an adaptive learning SaaS expected to operate for the next 10 years.

Task 005 — Design the Complete Domain Events, Commands, Queries & State Machines (No Application Code)

Context

The following documents are complete and authoritative:

- Task 001 — Architecture Specification
- Task 002 — Ubiquitous Language
- Task 003 — ADR Repository
- Task 004 — PostgreSQL Database Architecture

This task must conform to all previous documents.

If conflicts exist, identify them before proceeding.

Objective

Design the complete behavioral model of the Mastery Engine.

This is NOT implementation.

This is NOT API design.

This is the business behavior of the entire platform.

Every meaningful change in the system must be represented as:

- Command
- Domain Event
- Query
- State Transition

The result should completely describe how the system behaves.

Deliverables

Create:

docs/domain-behavior/

Containing:

README.md
01-commands.md
02-domain-events.md
03-queries.md
04-state-machines.md
05-workflows.md
06-sequence-diagrams.md
07-event-catalog.md
08-event-versioning.md
09-eventual-consistency.md
10-error-handling.md
11-idempotency.md
12-future-evolution.md

Part 1 — Commands

Define every command in the system.

For every command include:

- Name
- Purpose
- Initiator
- Preconditions
- Validation
- Business Rules
- Success Result
- Failure Cases
- Events Produced

Examples:

RegisterUser

LoginUser

CreateSubject

CreateConcept

PublishConcept

GenerateAdaptiveQueue

StartLearningSession

SubmitAnswer

RequestHint

CompleteSession

UpdateMastery

ScheduleReview

PublishQuestionTemplate

ArchiveContent

UpgradeSubscription

GrantAchievement

ImportContentPack

ExportLearnerData

DeleteUser

ResetMastery

...

Target:

100+ commands.

Part 2 — Domain Events

Define every event.

For every event:

- Name

- Trigger
- Payload
- Producer
- Consumers
- Ordering Requirements
- Retry Policy
- Idempotency Requirements
- Audit Requirements

Examples:

UserRegistered

LearningSessionStarted

QuestionAnswered

AttemptRecorded

MemoryUpdated

MasteryUpdated

ReviewScheduled

RecommendationGenerated

ConceptGraduated

AchievementUnlocked

NotificationQueued

SubscriptionActivated

AlgorithmVersionPublished

ContentPublished

...

Target:

150+ events.

Part 3 — Queries

Every read operation.

Examples:

GetDashboard

GetAdaptiveQueue

GetConceptProgress

GetWeakConcepts

GetAttemptHistory

GetLearningVelocity

GetRetentionAnalytics

GetInterviewReadiness

SearchConcepts

SearchQuestions

GetNotifications

...

Include:

Purpose

Input

Output

Consistency Requirements

Expected Latency

Caching Strategy

Authorization

Part 4 — State Machines

Design complete state machines.

Include diagrams.

State machines for:

Learning Session

Attempt

Question

Question Template

Concept

Content Version

Template Version

Algorithm Version

Subscription

Achievement

Review

Study Plan

Recommendation

Notification

User Account

Organization

Tenant

Feature Flag

Each state must define:

Entry

Exit

Allowed transitions

Invalid transitions

Triggers

Recovery

Part 5 — Business Workflows

Design end-to-end workflows.

Examples:

New learner onboarding

Daily learning

Adaptive queue generation

Question answering

Mastery recalculation

Review scheduling

Content publication

Algorithm rollout

Subscription upgrade

Organization creation

Subject creation

Achievement unlock

GDPR deletion

Backup restore

Disaster recovery

Part 6 — Sequence Diagrams

Produce Mermaid sequence diagrams.

Minimum:

30 diagrams.

Examples:

User Registration

Daily Learning Loop

Adaptive Scheduling

Mastery Update

Review Scheduling

Notification Flow

Content Publishing

Authentication

Recommendation Generation

Algorithm Upgrade

Part 7 — Event Catalog

Create a complete catalog.

Group events by bounded context.

Show:

Producer

Consumers

Ordering

Criticality

Retention

Replay support

Part 8 — Event Versioning

Explain:

Schema evolution

Backward compatibility

Replay

Deprecation

Migration

Version numbering

Part 9 — Eventual Consistency

Explain where the system is:

Strongly consistent

Eventually consistent

Synchronous

Asynchronous

Explain why.

Include Outbox integration.

Part 10 — Error Handling

Design business failures.

Examples:

Duplicate attempts

Late submissions

Deleted content

Expired sessions

Algorithm mismatch

Version conflicts

Concurrent mastery updates

Outbox failures

Notification failures

Recovery strategies.

Part 11 — Idempotency

Define idempotency strategy.

Commands

Events

Retries

Message processing

Duplicate prevention

Replay behavior

Idempotency keys

Part 12 — Future Evolution

Explain how the behavioral model evolves to support:

Machine Learning

Collaborative learning

Peer review

AI tutoring

Offline mode

Mobile sync

Marketplace

Enterprise organizations

Real-time classrooms

Acceptance Criteria

The document should allow engineers to implement every business workflow without ambiguity.

Every mutation must originate from a Command.

Every Command must produce one or more Domain Events.

Every Query must be read-only.

Every aggregate must have a documented lifecycle.

Every event must be replayable where appropriate.

Think like a Principal Domain Architect designing an event-driven adaptive learning platform that will operate for the next decade.

Task 006 — Design the Complete OpenAPI

3.1 API Contract (No Backend Implementation)

Context

The following documents are complete and are the authoritative source of truth:

- Task 001 — Architecture Specification
- Task 002 — Ubiquitous Language
- Task 003 — ADR Repository
- Task 004 — PostgreSQL Database Design
- Task 005 — Domain Behavior (Commands, Events, Queries, State Machines)

This task **MUST** conform to those documents.

The API is a contract.

It is **NOT** an implementation.

Do **NOT** write FastAPI code.

Do **NOT** write business logic.

Do **NOT** generate controllers.

Only design the API specification.

Objective

Produce a production-grade OpenAPI 3.1 contract for the Mastery Engine.

The contract should be sufficient for:

- Backend implementation
- Frontend development
- SDK generation
- API testing
- Documentation
- Mock server generation

without requiring application code.

Deliverables

Create:

`docs/api/`

Containing:

```
README.md
01-api-principles.md
02-resource-model.md
03-openapi-spec.yaml
04-authentication.md
05-error-model.md
06-pagination-filtering.md
07-versioning.md
08-idempotency.md
09-webhooks.md
10-rate-limiting.md
11-security.md
12-examples.md
13-sdk-guidelines.md
14-testing.md
15-future-evolution.md
```

Part 1 — API Principles

Document:

- REST principles
- Resource-oriented design

- Stateless requests
 - Idempotent operations
 - Consistent naming
 - JSON conventions
 - HTTP status code philosophy
-

Part 2 — Resource Model

Identify every API resource.

Minimum resources:

Authentication

Users

Profiles

Organizations

Tenants

Subjects

Learning Paths

Concepts

Learning Objectives

Misconceptions

Question Templates

Question Instances

Study Sessions

Adaptive Queues

Attempts

Reviews

Mastery Scores

Recommendations

Achievements

Milestones

Notifications

Subscriptions

Billing

Content Packs

Algorithm Versions

Feature Flags

Audit Logs

System Settings

Search

Analytics

Admin

Each resource should define:

Purpose

Ownership

Relationships

Allowed operations

Authorization

Part 3 — OpenAPI Specification

Produce a complete OpenAPI 3.1 YAML document.

Include:

Servers

Security schemes

Schemas

Components

Tags

Examples

Parameters

Request bodies

Responses

Headers

Callbacks

Links

Reusable objects

Every endpoint must reference reusable schemas.

Endpoint Design

Design endpoints for every command and query defined in Task 005.

Examples:

POST /auth/register

POST /auth/login

POST /study-sessions

GET /adaptive-queue

POST /attempts

GET /mastery

GET /recommendations

POST /content/question-templates

POST /algorithm-versions

...

Every endpoint should include:

Purpose

HTTP method

Request schema

Response schema

Possible errors

Authorization

Idempotency requirements

Part 4 — Authentication

Document:

JWT flow

Refresh tokens

Logout

Password reset

Email verification

MFA support

OAuth readiness

Token expiration

Session management

Part 5 — Error Model

Design a standard error object.

Include:

Error code

Message

Details

Correlation ID

Timestamp

Documentation link

Retry hint

Validation errors

Part 6 — Pagination & Filtering

Support:

Cursor pagination

Sorting

Filtering

Searching

Field selection

Expansion

Part 7 — API Versioning

Document:

URI vs Header versioning

Deprecation

Backward compatibility

Sunset policy

Breaking changes

Part 8 — Idempotency

Define:

Idempotency-Key header

Replay behavior

Duplicate request handling

Expiration

Conflict responses

Part 9 — Webhooks

Design webhook contracts for:

Achievement unlocked

Subscription updated

Organization events

Content published

Algorithm version released

Notification delivered

Retry strategy

Signing

Verification

Part 10 — Rate Limiting

Per-user

Per-IP

Per-tenant

Burst

Admin exemptions

Retry-After behavior

Part 11 — Security

OWASP API Security alignment.

Document:

Authorization

Mass assignment prevention

Object-level authorization

Input validation

Sensitive field handling

Audit logging

CORS

CSRF strategy

Part 12 — Examples

Provide complete example requests and responses for major workflows:

Registration

Login

Start study session

Retrieve adaptive queue

Submit answer

View mastery

Review session

Content publication

Admin operations

Part 13 — SDK Guidelines

Document conventions for future SDK generation:

TypeScript

Python

Kotlin

Swift

Naming

Error handling

Retries

Pagination

Part 14 — Testing

Define:

Contract testing

Schema validation

Mock server generation

Consumer-driven contracts

Backward compatibility tests

Part 15 — Future Evolution

Explain how the API evolves to support:

GraphQL

gRPC

Streaming

Server-Sent Events

Real-time collaboration

Offline sync

AI-assisted clients

Mobile-first optimizations

Acceptance Criteria

The API contract should allow frontend engineers and backend engineers to work independently.

Every endpoint must map to the Commands and Queries defined in Task 005.

Every schema must align with the PostgreSQL data model from Task 004.

The OpenAPI specification should be valid OpenAPI 3.1 and suitable for generating SDKs and mock servers.

Think like a Principal API Architect designing the public and internal API surface of a SaaS platform expected to operate for the next decade.

Task 007 — Build the Production Repository Foundation (Repository Scaffolding)

Context

The architecture and design phases are complete.

The following documents are now the source of truth:

- Task 001 — Architecture Specification
- Task 002 — Ubiquitous Language
- Task 003 — ADR Repository
- Task 004 — Database Architecture
- Task 005 — Domain Behavior
- Task 006 — OpenAPI Contract

From this point onward, implementation **MUST** conform to these documents.

No shortcuts.

No architectural changes unless accompanied by a new ADR.

Objective

Create the production repository structure for the Mastery Engine.

Do NOT implement business logic.

Do NOT implement endpoints.

Do NOT implement UI screens.

Only create the engineering foundation.

Everything should compile, lint, test, and run as an empty production application.

Repository Layout

Create a production-ready monorepo:

```
mastery-engine/  
├── backend/  
├── frontend/  
├── docs/  
├── infrastructure/  
├── packages/  
├── scripts/  
├── tests/  
├── .github/  
  
├── docker-compose.yml  
├── Makefile  
├── README.md  
├── LICENSE  
├── .editorconfig  
├── .gitignore  
├── .env.example  
└── .pre-commit-config.yaml
```

Backend

Create a FastAPI project using Clean Architecture.

Structure:

```
backend/
├── app/
│   ├── domain/
│   ├── application/
│   ├── infrastructure/
│   ├── presentation/
│   ├── shared/
│   └── main.py
├── tests/
├── alembic/
├── pyproject.toml
├── ruff.toml
├── mypy.ini
└── README.md
```

Requirements:

- Python 3.13
- FastAPI
- SQLAlchemy 2.x
- Alembic
- Pydantic v2
- Uvicorn
- Pytest
- httpx
- Ruff
- MyPy
- Structlog
- Dependency Injector (or equivalent)

The application should start successfully and expose only:

- /health
- /ready
- /live

No business endpoints yet.

Frontend

Create a Next.js project.

Requirements:

- Next.js (latest stable)
- TypeScript
- Tailwind CSS
- App Router
- ESLint
- Prettier
- React Query
- Zod
- React Hook Form

Suggested structure:

```
frontend/  
├── app/  
├── components/  
├── features/  
├── lib/  
├── hooks/  
├── providers/  
├── styles/  
├── types/  
└── public/
```

Only implement:

- Home page
- Health check page
- Basic layout
- Authentication provider placeholder

No feature pages.

Infrastructure

Create:

```
infrastructure/  
|
```

```
|— docker/  
|— nginx/  
|— postgres/  
|— redis/  
|— monitoring/
```

Include:

- Dockerfiles
 - Development compose configuration
 - PostgreSQL service
 - Redis service
 - Nginx reverse proxy
 - Prometheus configuration
 - Grafana provisioning
-

CI/CD

Create GitHub Actions:

- Lint
 - Unit Tests
 - Build
 - Security Scan
 - Dependency Audit
 - Docker Build
-

Engineering Standards

Configure:

- Ruff
 - Black (if used)
 - MyPy
 - ESLint
 - Prettier
 - EditorConfig
 - Pre-commit hooks
-

Logging

Configure structured logging using Structlog.

Every request should include:

- Request ID
 - Correlation ID
 - Timestamp
 - Duration
-

Configuration

Implement environment-based configuration.

Support:

- Development
- Testing
- Staging
- Production

Provide:

- `.env.example`
 - Configuration validation
 - Secrets separation
-

Testing

Create the testing structure only.

No business tests.

Verify:

- Backend starts
- Frontend builds
- Docker Compose starts

- Health endpoints respond
-

Documentation

Create:

- Root README
 - Backend README
 - Frontend README
 - Infrastructure README
 - Development Setup Guide
 - Contribution Guide
-

Acceptance Criteria

The repository should:

- Build successfully
- Pass linting
- Pass type checking
- Pass formatting checks
- Start with Docker Compose
- Expose health endpoints
- Support hot reload in development
- Be ready for implementation of business features

Do not implement business logic.

Do not implement database models.

Do not implement authentication.

Do not implement API endpoints beyond health checks.

Think like a Staff Platform Engineer creating the foundation for a production SaaS that will be developed over the next decade.

Task 008 — Implement the Complete Domain Layer

Context

The following artifacts are complete and authoritative:

- Task 001 — Architecture Specification
- Task 002 — Ubiquitous Language
- Task 003 — ADR Repository
- Task 004 — Database Design
- Task 005 — Domain Behavior
- Task 006 — OpenAPI Contract
- Task 007 — Repository Foundation

This task implements ONLY the Domain Layer.

No infrastructure.

No FastAPI routes.

No SQLAlchemy models.

No Alembic migrations.

No HTTP.

No Redis.

No external services.

The domain must remain pure Python.

Objective

Create the complete domain model for the Mastery Engine following Domain-Driven Design.

Everything in this task must be framework-independent.

The domain must be fully testable without a database or web server.

Implement

For every bounded context, create:

- Entities
 - Value Objects
 - Aggregates
 - Repository Interfaces
 - Domain Services (interfaces only where appropriate)
 - Domain Events
 - Domain Exceptions
 - Specifications (if useful)
 - Factory methods
 - Invariants
-

Entities

Implement entities such as:

User

Profile

Organization

Subject

LearningPath

Concept

ConceptDependency

LearningObjective

Misconception

QuestionTemplate

QuestionInstance

StudySession

AdaptiveQueue

Attempt

Review

MasteryScore

Recommendation

Achievement

Subscription

ContentPack

AlgorithmVersion

Notification

Every entity must:

- Have identity
- Encapsulate behavior
- Protect invariants
- Avoid anemic models

Value Objects

Implement immutable value objects including (as applicable):

Email

UserId

ConceptId

AttemptId

QuestionId

MasteryValue

MemoryValue

Confidence

Difficulty

ReviewInterval

Duration

Percentage

VersionNumber

CorrelationId

RequestId

Money

DateRange

LearningState

Use frozen dataclasses or equivalent immutable structures.

Aggregates

Identify aggregate roots and enforce boundaries.

Examples:

User Aggregate

StudySession Aggregate

Content Aggregate

Algorithm Aggregate

No aggregate may directly modify another aggregate.

Repository Interfaces

Define abstract interfaces only.

Examples:

UserRepository

ConceptRepository

AttemptRepository

MasteryRepository

QueueRepository

QuestionRepository

StudySessionRepository

No implementation.

No SQL.

No ORM.

Domain Events

Implement immutable domain events for major state transitions.

Examples:

AttemptSubmitted

MasteryUpdated

StudySessionStarted

StudySessionCompleted

RecommendationGenerated

AchievementUnlocked

ContentPublished

AlgorithmActivated

These are domain events, not integration events.

Domain Services

Implement only when behavior does not naturally belong to an entity.

Examples:

MasteryCalculator

RecommendationPolicy

QueueGenerator

SchedulingPolicy

AlgorithmSelector

As abstract interfaces or pure domain logic where appropriate.

Exceptions

Implement a clear hierarchy:

DomainError

InvariantViolation

BusinessRuleViolation

EntityNotFound

InvalidStateTransition

DuplicateAttempt

etc.

Invariants

Every aggregate must enforce invariants internally.

Examples:

Attempt cannot be modified after submission.

Completed StudySession cannot receive new Attempts.

MasteryScore must remain between 0 and 100.

MemoryScore must remain between 0 and 100.

Question difficulty must be valid.

Recommendation cannot reference archived content.

Tests

Write comprehensive unit tests for the domain layer.

Test:

Entity behavior

Value object validation

Aggregate invariants

Factories

Equality

Immutability

Domain events

Repository contracts (interface expectations)

No mocks of databases.

No HTTP.

No external dependencies.

Documentation

Produce:

docs/domain-model/

README.md

aggregate-map.md

entity-catalog.md

value-object-catalog.md

domain-event-catalog.md

repository-contracts.md

Acceptance Criteria

The domain layer:

- contains no framework imports
- contains no persistence logic
- contains no web logic
- contains no infrastructure dependencies

Every business rule from Tasks 001–006 must live here or be explicitly delegated to a domain service.

The domain layer should compile independently, be highly testable, and serve as the foundation for all future application services.

Think like an expert DDD architect building a domain model that should remain stable for the next decade.

Task 009 — Implement the Complete Application Layer

Context

The following are complete and authoritative:

- Task 001 — Architecture Specification
- Task 002 — Ubiquitous Language
- Task 003 — ADR Repository
- Task 004 — Database Design
- Task 005 — Domain Behavior
- Task 006 — OpenAPI Contract
- Task 007 — Repository Foundation
- Task 008 — Domain Layer

The Domain Layer is now frozen.

The Application Layer must orchestrate domain behavior without containing business rules that belong inside entities or domain services.

Objective

Implement the complete Application Layer following Clean Architecture.

This layer coordinates:

- Commands
- Command Handlers
- Queries
- Query Handlers
- DTOs

- Mappers
- Unit of Work abstraction
- Event Publisher abstraction
- Validation
- Transactions
- Authorization interfaces
- Application Services

No infrastructure implementations.

No FastAPI.

No SQLAlchemy.

No Redis.

No HTTP.

Only orchestration.

Folder Structure

```
app/application/
```

```
shared/  
commands/  
queries/  
dto/  
handlers/  
validators/  
services/  
ports/  
mappers/  
exceptions/  
unit_of_work.py  
event_publisher.py
```

Organize further by bounded context where appropriate.

Commands

Implement every coarse-grained command defined in Task 005.

Examples include:

RegisterUser

VerifyEmail

AuthenticateUser

EnrollLearner

CreateLearningGoal

StartStudySession

PauseStudySession

ResumeStudySession

EndStudySession

GenerateAdaptiveQueue

SubmitAttempt

UpdateMastery

ScheduleReview

GenerateRecommendation

UnlockAchievement

PublishContent

CreateQuestionTemplate

ReleaseAlgorithmVersion

CreateOrganization

EnableFeatureFlag

CreateSubscription

CancelSubscription

CreateNotification

...and the remaining commands from the domain behavior specification.

Each command should be immutable.

Command Handlers

Each command has exactly one handler.

Responsibilities:

- Validate command
- Load aggregates through repository interfaces
- Call domain behavior
- Persist through Unit of Work abstraction
- Collect domain events
- Publish through Event Publisher abstraction
- Return application result DTO

Handlers must never contain business rules already modeled in the domain.

Queries

Implement every query from Task 005.

Examples:

GetDashboard

GetMastery

GetAdaptiveQueue

GetRecommendations

GetStudySession

GetAttemptHistory

GetAchievements

GetLearningProgress

GetContentPack

GetAlgorithmVersion

GetNotifications

GetSubscription

...and all remaining read models.

Queries are read-only.

Query Handlers

Responsibilities:

- Read repositories
 - Build DTOs
 - Apply authorization checks
 - Never modify state
-

DTOs

Separate:

Command DTOs

Response DTOs

Read Models

Summary Models

Detail Models

Never expose domain entities directly.

Mappers

Create dedicated mappers between:

Domain $\rightarrow$ DTO

DTO $\rightarrow$ Domain Inputs

No mapping logic inside handlers.

Validation

Implement application validators.

Examples:

RegisterUserValidator

SubmitAttemptValidator

PublishContentValidator

StartStudySessionValidator

Only validate:

- shape
- presence
- application-level constraints

Business invariants remain inside the domain.

Unit of Work

Define interface only.

Responsibilities:

begin()

commit()

rollback()

collect_events()

Repository access

No implementation.

Event Publisher

Define interface only.

publish()

publish_many()

No infrastructure.

Authorization

Define interfaces.

Examples:

AuthorizationService

PermissionChecker

CurrentUserProvider

No implementation.

Transactions

Command handlers should clearly define transaction boundaries using the Unit of Work abstraction.

No database-specific code.

Application Services

Implement orchestration services where a workflow spans multiple handlers.

Examples:

LearningApplicationService

ContentApplicationService

IdentityApplicationService

AdministrationApplicationService

Keep these thin.

Exceptions

Implement application-specific exceptions:

ValidationFailed

AuthorizationDenied

ConcurrencyConflict

ResourceMissing

ApplicationConflict

Do not reuse domain exceptions for application concerns.

Tests

Create comprehensive tests.

Test:

Command handlers

Query handlers

DTO mapping

Validation

Transaction orchestration

Unit of Work interaction

Event publication

Use fake repositories and fake Unit of Work implementations.

Never use a database.

Never use FastAPI.

Documentation

Create:

docs/application/

README.md

command-catalog.md

query-catalog.md

application-services.md

transaction-boundaries.md

Acceptance Criteria

The Application Layer:

- depends only on the Domain Layer
- contains no infrastructure
- contains no web framework
- contains no ORM
- contains no SQL
- contains no HTTP

Every write workflow is implemented through commands.

Every read workflow is implemented through queries.

Every transaction passes through the Unit of Work abstraction.

Every domain event flows through the Event Publisher abstraction.

The entire layer should be independently testable using fake repositories.

Think like a Principal Engineer implementing the orchestration layer for a large-scale SaaS that will evolve for the next decade.

Task 010 — Implement the Complete Infrastructure Layer

Context

The following are complete and authoritative:

- Task 001 — Architecture Specification
- Task 002 — Ubiquitous Language
- Task 003 — ADR Repository
- Task 004 — Database Design
- Task 005 — Domain Behavior
- Task 006 — OpenAPI Contract
- Task 007 — Repository Foundation
- Task 008 — Domain Layer

- Task 009 — Application Layer

The Domain Layer and Application Layer are frozen.

Infrastructure must implement their interfaces without changing their contracts.

Objective

Implement the complete Infrastructure Layer.

This layer connects the Application Layer to PostgreSQL and external systems while preserving Clean Architecture.

No FastAPI controllers.

No frontend.

No authentication.

Only infrastructure.

Folder Structure

```
app/infrastructure/
```

```
database/  
orm/  
repositories/  
mappers/  
unit_of_work/  
events/  
outbox/  
cache/  
config/  
logging/  
clock/  
ids/
```

Database

Implement:

- Async SQLAlchemy 2.x
- Engine factory
- Session factory
- Connection pool configuration
- Retry strategy
- Health checks
- Database lifecycle

Support:

- Development
 - Testing
 - Production
-

ORM Models

Implement SQLAlchemy models for every table defined in Task 004.

Requirements:

- Proper relationships
- Foreign keys
- Constraints
- Composite indexes
- Version columns
- UUID v7 support
- JSONB support
- Soft delete where required
- Append-only enforcement where required

ORM models are persistence models only.

They are NOT domain entities.

Domain Mappers

Create bidirectional mappers:

Domain Entity $\rightleftharpoons$ ORM Model

No ORM leakage into the Domain Layer.

Repository Implementations

Implement every repository interface created in Task 008.

Examples:

UserRepository

AttemptRepository

ConceptRepository

QuestionRepository

StudySessionRepository

MasteryRepository

QueueRepository

ReviewRepository

NotificationRepository

etc.

Repositories should:

- return domain entities
- persist aggregates
- support pagination
- support optimistic concurrency
- never expose ORM models

Unit of Work

Implement AsyncUnitOfWork.

Responsibilities:

- transaction begin
 - commit
 - rollback
 - repository lifetime
 - event collection
 - event persistence
 - event publication after successful commit
-

Outbox Pattern

Implement transactional outbox.

Requirements:

- Outbox table integration
 - Store domain events in same transaction
 - Publish after commit
 - Retry failed events
 - Idempotency support
 - Replay support
-

Event Serialization

Implement serializers for domain events.

Requirements:

- JSON
 - version field
 - metadata
 - correlation id
 - timestamp
 - aggregate id
 - event type
-

Alembic

Generate:

- initial migration
- migration environment
- naming conventions
- migration helpers

Migration must exactly match Task 004.

Query Optimization

Implement:

- eager loading strategy
 - lazy loading policy
 - pagination helpers
 - batch loading
 - read models where appropriate
-

Concurrency

Implement:

- optimistic locking
 - retry strategy
 - conflict exceptions
 - compare-and-swap updates
-

Caching

Implement interfaces for:

- Redis cache
- local cache

No application logic should depend on Redis directly.

Logging

Implement infrastructure logging.

Include:

- SQL timing
 - slow query logging
 - transaction ids
 - connection ids
-

Clock

Create injectable clock abstraction.

Avoid direct `datetime.now()` inside infrastructure code.

ID Generation

Implement UUID v7 generation service.

Support deterministic IDs in tests.

Tests

Create:

- repository integration tests
- mapper tests
- transaction tests
- concurrency tests
- migration tests

Use PostgreSQL containers.

No mocks.

Documentation

Create:

docs/infrastructure/

README.md

repository-implementation.md

mapping-strategy.md

unit-of-work.md

outbox-pattern.md

migration-guide.md

performance-notes.md

Acceptance Criteria

Infrastructure:

- implements every repository contract
- persists every aggregate
- hides SQLAlchemy completely
- supports optimistic concurrency
- supports append-only tables
- implements transactional outbox
- passes integration tests
- supports Alembic migrations

No business rules should move into infrastructure.

Infrastructure exists only to support the domain.

Think like a Staff Backend Engineer building the persistence layer for a SaaS expected to run reliably for the next 10 years.

Task 011 — First Vertical Slice: Learner Onboarding & Session Bootstrap

Context

Tasks 001–010 are complete.

The architecture, domain, application, and infrastructure layers are frozen except for bug fixes.

This task must integrate those layers into the first fully working feature.

Objective

Implement the first end-to-end workflow:

1. Register a learner
2. Verify email (development mode may auto-verify)
3. Enroll in the Python Interview Preparation subject
4. Create a learning goal
5. Start a study session
6. Generate the initial adaptive queue
7. Return the queue to the client

No attempt submission yet.

No mastery updates yet.

Those belong to the next slice.

Scope

Implement the complete path from HTTP request to database and back.

The feature must exercise:

- FastAPI
 - Application Layer
 - Domain Layer
 - Infrastructure Layer
 - PostgreSQL
 - Transaction handling
 - Repository implementations
 - Event publishing
 - OpenAPI contract
-

FastAPI

Implement the API endpoints defined in Task 006 for:

- Register User
- Verify Email
- Enroll Learner
- Create Learning Goal
- Start Study Session
- Get Adaptive Queue

Generate the routes directly from the OpenAPI contract.

No contract drift.

Authentication

Development mode:

- JWT support enabled
- Email verification may be bypassed with a feature flag

Production mode:

- Follow the API contract exactly
-

Adaptive Queue (Version 1)

Implement the deterministic scheduler only.

Do NOT use ML.

Queue generation should consider:

- Learning goal
- Concept prerequisites
- Difficulty progression
- Previously completed concepts
- Existing recommendations
- Daily limits

The output should be stable for identical inputs.

Queue Composition

Each queue item should contain:

- Question Instance ID
 - Concept ID
 - Difficulty
 - Estimated Duration
 - Recommendation Score
 - Reason (e.g. "new concept", "review due")
-

Transactions

Each write request:

- One Unit of Work
 - One commit
 - Outbox event creation
 - Domain events persisted
 - Events published after commit
-

API Responses

Return DTOs only.

Never serialize domain entities.

Responses must match Task 006 schemas exactly.

Integration

Wire together:

FastAPI

- Application Handler
 - Repository
 - Domain
 - Repository
 - Unit of Work
 - Outbox
 - Response DTO
-

Testing

Create:

Unit Tests

- Command handlers
- Queue generation
- Validation

Integration Tests

Using PostgreSQL:

- Register user
- Enroll learner
- Start session
- Generate queue

API Tests

Using httpx/TestClient:

- Happy path
 - Validation failures
 - Authorization failures
 - Duplicate enrollment
 - Session already active
-

Observability

Every request should log:

- Request ID
 - Correlation ID
 - User ID
 - Session ID
 - Transaction duration
-

Documentation

Create:

docs/vertical-slices/

01-onboarding.md

Include:

- sequence diagram
 - request flow
 - events emitted
 - transaction boundaries
 - failure scenarios
-

Acceptance Criteria

A developer can:

1. Start Docker Compose
2. Open Swagger UI
3. Register a user
4. Enroll in Python Interview Prep
5. Start a study session
6. Receive a deterministic adaptive queue

The entire workflow must execute against PostgreSQL using the real repositories.

No mocks.

No fake repositories.

No manual SQL.

Think like a Staff Backend Engineer delivering the first production-ready feature that proves the architecture is viable.

MASTERY ENGINE

TASK 012 — COMPLETE LEARNING LOOP VERTICAL SLICE

You are the Principal Software Architect and Lead Engineer of the Mastery Engine project.

You have already completed Tasks 001–011.

DO NOT redesign architecture.

DO NOT change terminology.

DO NOT rename entities.

Everything MUST remain consistent with:

- Architecture Specification

- ADR Repository
- Database Design
- Domain Behavior
- OpenAPI Contract
- Domain Layer
- Application Layer
- Infrastructure Layer
- Vertical Slice 01

Those documents are the source of truth.

MISSION

Build the SECOND production vertical slice.

This slice completes the actual learning loop.

Current status:

User can

✓ Register

✓ Verify Email

✓ Login

✓ Enroll

✓ Create Goal

✓ Start Session

✓ Receive Adaptive Queue

But cannot actually study.

This task fixes that.

GOAL

The learner must be able to complete one entire learning cycle.

Adaptive Queue

↓

Question



Submit Answer



Create Attempt



Update Mastery



Schedule Review



Return Explanation



Next Question

Everything must be fully integrated through:

FastAPI



Application



Domain



Infrastructure



PostgreSQL

No mocks.

No fake repositories.

Real repositories.

Real Unit of Work.

Real events.

IMPLEMENT

=====

PART 1

Question Retrieval

=====

Implement

GET

/api/v1/questions/{question_instance_id}

Returns

- question

- options
- code snippet
- metadata
- concept ids
- difficulty
- estimated duration

QuestionInstance comes from database.

Must respect Content Version.

Must respect Template Version.

Must never expose Correct Answer.

=====

PART 2

Answer Submission

=====

Implement

POST

/api/v1/questions/{question_instance_id}/submit

Request

answer

confidence

time_spent_seconds

hint_used

Response

Attempt Result

Memory Score

Mastery Score

Review Date

Explanation

Recommendation

This endpoint must execute

SubmitAttemptHandler

which already exists.

=====

PART 3

Mastery Update

=====

Wire

UpdateMasteryHandler

into the request pipeline.

Current flow

Attempt



Application



Mastery Handler



Mastery Calculator



Repository



Outbox



Commit

Everything must happen automatically.

PART 4

Review Scheduling

When mastery updates

Generate

or

Update

Review.

Rules come from ADR-0007.

No ML.

Deterministic.

Intervals

Today

Tomorrow

3 Days

7 Days

14 Days

30 Days

based on

Memory Score

Mastery Score

Confidence

Response Time

=====

PART 5

Explanation Delivery

=====

Return explanation from database.

Do NOT call any LLM.

Explanation already belongs to

Concept

Template Version

or

Question Template.

Explanation selection rules

Correct

↓

Short reinforcement

Incorrect



Detailed explanation

Hint Used



Expanded explanation

=====

PART 6

Recommendation Engine

=====

After submission

Return recommendation

Examples

Review Lists

Practice Dictionaries

Retry Variable Scope

Advance to Functions

Recommendation comes from

Recommendation Aggregate

No AI.

Rule based.

=====

PART 7

Dashboard Update

=====

GET

/api/v1/dashboard

must now include

Current Mastery

Weak Concepts

Strong Concepts

Today's Reviews

Today's Queue

Daily Progress

Current Streak

Memory Trend

Mastery Trend

Interview Readiness

=====

PART 8

Events

=====

Ensure events flow correctly

AttemptRecorded

↓

MasteryUpdated

↓

ConceptStateChanged

↓

ReviewScheduled

↓

RecommendationGenerated

↓

DashboardUpdated

Every event

written

Outbox

committed

published

=====

PART 9

Repositories

=====

Implement remaining repository methods required

Question Repository

Recommendation Repository

Review Repository

Mastery Repository

No TODOs.

=====

PART 10

Tests

=====

Integration Tests

Register

Login

Enroll

Goal

Session

Queue

Retrieve Question

Submit Correct Answer

Submit Wrong Answer

Hint Used

Confidence Low

Confidence High

Fast Response

Slow Response

Review Scheduled

Mastery Updated

Dashboard Updated

Recommendation Returned

Verify

database

events

responses

mastery values

review date

=====

PART 11

Documentation

=====

Create

docs/vertical-slices/02-learning-loop.md

Include

Sequence Diagram

Learning Loop Diagram

Transaction Boundaries

Events

Failure Cases

State Changes

API Examples

Performance Notes

Future Extension Points

QUALITY RULES

No business logic inside FastAPI.

No ORM leakage.

No SQL inside handlers.

Domain remains pure.

Repositories hide persistence.

Application orchestrates.

Infrastructure persists.

Presentation validates HTTP.

Everything follows Clean Architecture.

OUTPUT

Generate

complete production implementation

not pseudocode.

No placeholders.

No TODO comments.

No "implement later."

Everything production ready.

ACCEPTANCE CRITERIA

I must be able to

Register

↓

Enroll



Start Session



Receive Queue



Open Question



Submit Answer



Attempt Saved



Mastery Updated

↓

Review Scheduled

↓

Explanation Returned

↓

Recommendation Returned

↓

Dashboard Updated

using only the running application.

That completes the first full end-to-end learning loop.

MASTERY ENGINE

TASK 013 — CONTENT FACTORY + QUESTION GENERATION SYSTEM

You are the Principal Software Architect and Lead Engineer of the Mastery Engine.

Tasks 001–012 are complete.

DO NOT redesign architecture.

DO NOT rename entities.

Everything MUST remain consistent with

- Architecture Specification
- ADR Repository
- Database Design
- Domain Behavior
- OpenAPI Contract
- Domain Layer
- Application Layer
- Infrastructure Layer
- Vertical Slice 01
- Vertical Slice 02

Those documents are the only source of truth.

MISSION

Implement the complete Content System.

This task replaces every placeholder used inside the learning loop.

The result should allow an administrator to build an entire Python Interview curriculum without touching code.

Everything becomes data.

GOAL

Support this pipeline

Subject

↓

Content Pack

↓

Concept

↓

Learning Objective



Misconception



Question Template



Template Version



Question Factory



Question Instance



Adaptive Queue



Attempt

PART 1

Content Administration API

Implement complete CRUD endpoints for

Subjects

Content Packs

Concepts

Learning Objectives

Misconceptions

Question Templates

Template Versions

Content Versions

All operations must use

Commands

Queries

Repositories

Unit of Work

No business logic inside controllers.

PART 2

Question Template Model

Every QuestionTemplate must support

Title

Purpose

Concepts

Difficulty

Estimated Time

Question Type

Question Stem Template

Variables

Constraints

Correct Answer Logic

Distractors

Explanation

Hint

Interview Notes

Tags

Version

Review Status

Author

Approval Metadata

PART 3

Question Factory

Implement

QuestionFactory

Responsibilities

Generate QuestionInstance

Generate deterministic variables

Render template

Generate answer choices

Generate metadata

Generate explanation reference

Generate seed

Store template version

Store content version

Store algorithm version

Same template

+

Same seed

↓

Must always generate identical QuestionInstance.

PART 4

Template Engine

Support placeholders such as

`{{variable}}`

`{{datatype}}`

`{{list}}`

`{{dictionary}}`

`{{function}}`

`{{class}}`

`{{number}}`

`{{operator}}`

Implement deterministic expansion.

No runtime LLM.

PART 5

Variable Generator

Create reusable generators

Random Integer

Random Float

Variable Name

Function Name

List

Tuple

Dictionary

String

Boolean

Nested Structure

Each generator accepts

Seed

Difficulty

Constraints

Returns deterministic values.

PART 6

Concept Mapping

Every QuestionTemplate may belong to

multiple Concepts.

Implement

TemplateConcept

Weight

Primary Concept

Secondary Concept

Dependency Depth

Importance

This replaces placeholder concept linkage.

PART 7

Explanation System

Implement Explanation Repository.

Explanation belongs to

Template Version

Concept

Difficulty

Scenario

Support

Correct Explanation

Incorrect Explanation

Hint Expansion

Interview Explanation

Beginner Explanation

No LLM.

PART 8

Distractor Library

Implement reusable distractors.

Distractor

Reason

Misconception

Difficulty

Usage Count

Success Rate

Enable adaptive distractor selection.

PART 9

Question Versioning

Every QuestionInstance stores

Template Version

Content Version

Algorithm Version

Generation Seed

Render Hash

Replay must reconstruct the exact question.

PART 10

Publishing Workflow

Draft



Review



Approved



Published



Archived

No learner can receive

Draft

Review

Archived

content.

PART 11

Import / Export

Support

JSON

Markdown

CSV

for

Subjects

Concepts

Templates

Content Packs

Version history

PART 12

Search

Implement

Full-text search

Filter by

Concept

Difficulty

Tag

Author

Version

Status

Question Type

PART 13

Analytics

Track

Template Usage

Success Rate

Average Time

Confidence

Hint Rate

Misconceptions Triggered

Weak Concept Coverage

PART 14

Integration

Replace placeholder Question IDs used by

Adaptive Queue.

Queue Generator must request

QuestionFactory



QuestionInstance



Database



Return real IDs

No random UUID placeholders remain.

PART 15

Testing

Integration tests

Create Subject

Create Concept

Create Template

Approve Template

Publish Template

Generate Question

Retrieve Question

Verify deterministic rendering

Same seed

↓

Same output

Different seed

↓

Different output

Verify version replay

Verify multiple concepts

Verify publishing workflow

Verify search

PART 16

Documentation

Create

docs/vertical-slices/03-content-system.md

Include

Architecture

Content Pipeline

Publishing Workflow

Question Factory

Versioning

Template Rendering

Deterministic Generation

Concept Mapping

Search

Analytics

Import Export

Future AI Authoring

QUALITY RULES

No runtime AI.

Everything deterministic.

No duplicate business logic.

Pure domain services.

Repositories hide persistence.

FastAPI only validates HTTP.

Application orchestrates.

Infrastructure persists.

ACCEPTANCE CRITERIA

An administrator must be able to

Create Subject

↓

Create Concept



Create Learning Objective



Create Misconception



Create Question Template



Approve



Publish



QuestionFactory generates QuestionInstance

↓

Adaptive Queue receives real Question IDs

↓

Learner studies

↓

Attempts reference actual Concepts

↓

Mastery updates correctly

↓

Replay reconstructs identical question.

This completely replaces every placeholder remaining in Tasks 011–012.

MASTERY ENGINE

TASK 014 — COMPLETE CONTENT FACTORY INTEGRATION

You are the Principal Software Architect of the Mastery Engine.

Tasks 001–013 are complete.

DO NOT redesign architecture.

DO NOT rename entities.

Everything must remain fully compatible with

- Architecture Specification
- ADRs
- Database Design
- Domain Layer
- Application Layer
- Infrastructure Layer
- OpenAPI
- Vertical Slice 01
- Vertical Slice 02
- Vertical Slice 03

MISSION

Remove every remaining placeholder from the runtime learning engine.

The Question Factory already exists.

The Content System already exists.

The Adaptive Queue already exists.

The Learning Loop already exists.

They now need to become ONE integrated system.

CURRENT PROBLEM

Today

Adaptive Queue

↓

creates placeholder Question IDs

↓

Submit Route



uses placeholder Concept IDs



Mastery updates



works

but does not use the real content pipeline.

After this task

everything must use

published content.

PART 1

Queue Generator Integration

Replace placeholder UUID generation.

Queue generation must become

Load published TemplateVersions



Filter by

Subject

Difficulty

Concept

Review status

Learning Goal



Select TemplateVersion



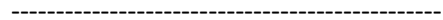
QuestionFactory.generate()



Persist QuestionInstance



Return real QuestionInstance IDs



PART 2

QuestionInstance Persistence

Every generated question must be stored.

Store

QuestionInstance

Generation Seed

Render Hash

Variables

Template Version

Content Version

Algorithm Version

Concept IDs

Rendered Prompt

Rendered Choices

Rendered Metadata

Generated At

PART 3

Question Retrieval

GET /questions/{id}

must now load

persisted QuestionInstance

instead of generating on demand.

Replay must never regenerate.

Always serve stored content.

PART 4

Submit Flow Integration

Remove placeholder

ConceptId(uuid4())

Load

QuestionInstance



TemplateVersion



TemplateConcept



Actual Concept IDs



Attempt.record()



Mastery update

Every attempt must reference

real concepts.

PART 5

Mastery Integration

UpdateMasteryHandler

must now consume

Attempt.concept_ids

instead of placeholder values.

One attempt

may update

multiple concepts

according to

TemplateConcept weights.

Support

Primary Concept

Secondary Concept

Weight

PART 6

Weighted Concept Updates

Implement

TemplateConcept weighting.

Example

Dictionary

weight 0.8

Hashing

weight 0.2

Memory

↓

80%

20%

Mastery updates accordingly.

PART 7

Explanation Integration

Explanation returned from

Explanation Repository

using

Template Version

Outcome

Difficulty

Scenario

Never dynamically built.

PART 8

Adaptive Queue Improvement

Queue selection now considers

Weak Concepts

Due Reviews

Learning Goal

Difficulty Progression

Already Served Questions

Recent Attempts

Template Usage

Question Freshness

Avoid duplicates.

PART 9

Replay Verification

Implement endpoint

GET

/admin/questions/{id}/replay

Verifies

Stored Question

↓

Replay QuestionFactory



Render Hash



Identical Output

Returns verification report.

PART 10

Analytics

Track

Template Usage

Question Usage

Replay Count

Generation Time

Average Difficulty

Generation Failures

Concept Coverage

PART 11

Caching

Cache

Published Template Versions

Content Versions

Concept Graph

Explanation Variants

Invalidate on publish.

PART 12

Tests

Integration tests

Queue creates real QuestionInstances

Question stored

Question retrieved

Submit updates real concepts

Weighted mastery update

Explanation loaded from repository

Replay verification passes

Duplicate prevention

Cache invalidation

PART 13

Documentation

Create

[docs/vertical-slices/04-content-integration.md](#)

Include

Architecture

Sequence Diagram

Queue Flow

Persistence Flow

Replay Flow

Mastery Flow

Concept Weighting

Caching

Performance

Failure Cases

QUALITY RULES

No placeholder UUIDs.

No fake concept IDs.

No runtime regeneration.

No duplicate questions.

No ORM leakage.

No business logic in FastAPI.

Pure domain services.

Deterministic generation.

ACCEPTANCE CRITERIA

When a learner starts a session



Adaptive Queue selects published templates



QuestionFactory generates real QuestionInstances



QuestionInstances are persisted



Learner opens stored questions



Learner submits answers



Attempts reference real concepts



Weighted mastery updates execute



Explanation loads from repository



Review scheduling executes



Replay verification proves deterministic generation



No placeholder IDs remain anywhere in the runtime pipeline.

Task 015 — Production Authentication, Authorization & Security Hardening

Objective

Replace every development-only authentication component with the production implementation described in ADR-0013, the API specification, and database design.

This task must NOT introduce placeholder implementations.

Every security decision must match previous documentation.

Deliverable

```
mastery-engine/  
  
backend/  
  app/  
    domain/  
    application/  
    infrastructure/  
    presentation/  
  
docs/  
  security/  
  authentication/  
  
tests/
```

Approximately **5,000–8,000 lines**.

Replace Development Implementations

The following **MUST** be removed completely.

Replace:

- SHA256 password hashing
- HS256 JWT
- email token == user id
- simplified login
- fake session validation
- simplified refresh flow

with production implementations.

Password Security

Implement

Argon2id

Requirements

- configurable memory cost
- configurable iterations
- configurable parallelism
- automatic rehash detection
- password upgrade on login
- constant-time verification

No SHA256 anywhere.

JWT

Replace HS256.

Use

RS256

Implement

Access Tokens

15 minute expiration

Refresh Tokens

30 day expiration

Key rotation support

kid header

private/public key loading

revocation support

token version support

issuer

audience

clock skew

Refresh Token Rotation

Implement

one-time refresh tokens

When refresh occurs

old refresh token becomes invalid

new one issued

reuse detection

session invalidation

security event generation

Sessions

Implement complete session model

Multiple devices

Device metadata

IP address

User agent

Creation

Last activity

Revocation

Logout

Logout all devices

Expiration

Idle timeout

Absolute timeout

Email Verification

Replace simplified implementation.

Implement

secure random verification tokens

expiration

single use

reissue

verification throttling

resend endpoint

Password Reset

Implement

forgot password

secure reset token

expiration

single use

token invalidation

password reset confirmation

session revocation after reset

MFA

Implement

TOTP

QR generation

Recovery codes

Enable

Disable

Verify

Backup codes

One-time usage

Recovery code regeneration

OAuth

Implement

Google

GitHub

Account linking

Existing user linking

New account creation

Verified email handling

Conflict resolution

Authorization

Implement complete RBAC.

Roles

Learner

Instructor

Administrator

Content Editor

Organization Admin

System Admin

Permissions

Fine-grained permission model

Object-level authorization

Ownership checks

Policy evaluation

Permission cache

Security Middleware

Implement

Request ID

Correlation ID

Security headers

CSP

HSTS

Referrer Policy

Permissions Policy

X-Frame-Options

X-Content-Type-Options

Trusted Host

HTTPS redirect

Rate Limiting

Implement

Redis token bucket

Per IP

Per User

Per Endpoint

Burst limits

Sliding window metrics

Retry headers

Whitelist

Admin bypass

CSRF

Implement

Double submit cookie

Origin validation

SameSite

Custom header validation

Secrets

Implement

Secret manager abstraction

Environment loader

Key rotation

Encryption key versioning

Startup validation

Audit Logging

Every security-sensitive action generates immutable audit logs.

Include

login

logout

password change

password reset

email verification

MFA enable

MFA disable

role changes

permission changes

OAuth linking

token refresh

session revoke

administrator actions

Domain Events

Implement

UserLoggedIn

UserLoggedIn
UserLoggedOut
PasswordChanged
PasswordResetRequested
PasswordResetCompleted
EmailVerificationRequested
EmailVerified
RefreshTokenRotated
SessionRevoked
SessionExpired
MfaEnabled
MfaDisabled
RecoveryCodesRegenerated
OAuthAccountLinked
RoleAssigned
PermissionGranted
PermissionRevoked
SecurityIncidentDetected

API Endpoints

Complete implementation of

POST /auth/register

POST /auth/login

POST /auth/logout

POST /auth/logout-all

POST /auth/refresh

POST /auth/verify-email

POST /auth/resend-verification

POST /auth/forgot-password

POST /auth/reset-password

POST /auth/change-password

POST /auth/mfa/setup

POST /auth/mfa/verify

POST /auth/mfa/enable

POST /auth/mfa/disable

POST /auth/mfa/recovery

GET /auth/sessions

DELETE /auth/sessions/{id}

GET /users/me

PATCH /users/me

GET /users/me/security

Infrastructure

Implement

Argon2 service

JWT service

RSA key loader

Session repository

Verification token repository

Password reset repository

OAuth service

Redis rate limiter

Permission cache

Audit repository

Documentation

Create

`docs/security/`

`authentication.md`

`authorization.md`

`jwt.md`

`mfa.md`

`oauth.md`

`password-policy.md`

`session-management.md`

`audit-logging.md`

`security-checklist.md`

`threat-model.md`

Testing

Implement

Unit Tests

Integration Tests

Security Tests

Property Tests

Concurrency Tests

JWT Tests

Argon2 Tests

OAuth Tests

Session Tests

MFA Tests

Permission Tests

Rate Limiter Tests

Replay Attack Tests

Refresh Rotation Tests

Password Reset Tests

Email Verification Tests

CSRF Tests

Audit Tests

Target

400 tests

Acceptance Criteria

- No SHA256 remains.
- No HS256 remains.
- RS256 implemented with key rotation.
- Argon2id implemented.
- Refresh token rotation operational.
- Sessions fully managed.
- MFA fully operational.
- OAuth (Google & GitHub) operational.
- Fine-grained RBAC implemented.

- Rate limiting enforced.
- CSRF protection enabled.
- Security headers configured.
- Audit logs generated for all sensitive operations.
- All endpoints match the OpenAPI contract from Task 006.
- All implementations align with ADR-0013 and previous architectural decisions.
- All tests pass with no mocked security-critical paths.

Task 016 — Production Authentication Vertical Slice

Objective

Replace every remaining development authentication endpoint with the production authentication pipeline built in Task 015.

This task fully integrates:

- FastAPI
- Application Layer
- Domain Layer
- Infrastructure Layer
- PostgreSQL
- Redis (where applicable)

No simplified authentication code may remain.

Deliverable

mastery-engine/

```
backend/  
  app/  
    presentation/  
    application/  
    domain/  
    infrastructure/
```

```
docs/  
  vertical-slices/  
    05-production-authentication.md
```

tests/

Expected size:

- ~4,000–7,000 lines

- 25–40 new files
 - 250+ tests
-

Replace Existing Auth Endpoints

Rewrite every endpoint.

Remove all remaining:

- SHA256 usage
- HS256 usage
- fake email verification
- simplified login
- fake sessions

Use the production services from Task 015.

Registration

Implement

POST /api/v1/auth/register

Flow

Validate request

↓

Argon2 hash password

↓

Create User aggregate

↓

Create UserCredential

↓

Create verification token



Persist transaction



Write outbox events



Return access token + refresh token



Return verification required state

Login

POST /api/v1/auth/login

Flow

Validate credentials



PasswordService.verify_and_upgrade()



Upgrade hash if needed



Create Session



Issue JWT



Issue Refresh Token



Store refresh hash



Write Audit Log



Publish UserLoggedIn

Refresh

POST /api/v1/auth/refresh

Requirements

Single-use refresh tokens

Reuse detection

Family revocation

Rotation

New access token

New refresh token

Session update

Audit log

Security event

Logout

POST /api/v1/auth/logout

Current device only

Revoke session

Invalidate refresh token

Publish event

Audit

Logout All

POST /api/v1/auth/logout-all

Revoke every session

Invalidate every refresh token

Publish SessionRevoked

Audit

Email Verification

Implement

POST /auth/verify-email

POST /auth/resend-verification

Requirements

Secure verification token

Single use

Expiration

Throttle resend

Verification audit

Password Reset

Implement

```
POST /auth/forgot-password
POST /auth/reset-password
POST /auth/change-password
```

Requirements

Token issuance

Expiration

Single use

Argon2 hashing

Session revocation

Refresh invalidation

Audit

MFA

Implement

```
POST /auth/mfa/setup
POST /auth/mfa/verify
POST /auth/mfa/enable
POST /auth/mfa/disable
POST /auth/mfa/recovery
```

Requirements

TOTP

Recovery codes

QR URI

Session validation

Audit

User Profile

Implement

```
GET /users/me  
PATCH /users/me  
GET /users/me/security
```

Include

Profile

Roles

Permissions

Sessions

MFA enabled

Password updated

Security history summary

Application Layer

Add missing handlers

Examples

```
RegisterUserHandler
```

```
LoginHandler
```

```
RefreshTokenHandler
```

```
LogoutHandler
```

LogoutAllHandler
VerifyEmailHandler
ResendVerificationHandler
ForgotPasswordHandler
ResetPasswordHandler
ChangePasswordHandler
SetupMfaHandler
EnableMfaHandler
DisableMfaHandler
VerifyMfaHandler
UseRecoveryCodeHandler
GetCurrentUserHandler
UpdateProfileHandler
GetSecurityDashboardHandler

Every write is a Command.

Every read is a Query.

Infrastructure

Wire together

PasswordService

JWTService

TokenService

SessionService

MFAService

AuthorizationService

Audit Repository

Outbox Writer

Redis Rate Limiter

RSA Key Loader

Database

Implement remaining authentication tables if missing

Examples

`verification_tokens`

`password_reset_tokens`

`refresh_tokens`

`sessions`

`mfa_recovery_codes`

`security_incidents`

Add migrations.

Audit Logging

Every authentication operation produces immutable audit records.

Examples

`LOGIN_SUCCESS`

`LOGIN_FAILURE`

`LOGOUT`

`LOGOUT_ALL`

`PASSWORD_CHANGED`

PASSWORD_RESET

EMAIL_VERIFIED

MFA_ENABLED

MFA_DISABLED

REFRESH_ROTATED

SESSION_REVOKED

SECURITY_INCIDENT

OpenAPI Alignment

Every endpoint must exactly match Task 006.

Request

Response

Errors

Status Codes

Security

Examples

Integration Tests

Implement

Registration

Duplicate email

Weak password

Login success

Login failure

Password upgrade

Refresh rotation

Refresh reuse detection

Logout

Logout all

Email verification

Resend verification

Password reset

Password change

MFA enable

MFA verify

Recovery codes

Current user

Update profile

Security dashboard

Expired tokens

Revoked sessions

Rate limiting

CSRF

JWT validation

Concurrent refresh

Documentation

Create

`docs/vertical-slices/`

`05-production-authentication.md`

Include

- Architecture diagram
 - Sequence diagrams
 - Authentication lifecycle
 - Session lifecycle
 - Refresh rotation flow
 - Password reset flow
 - MFA flow
 - Security event flow
 - Audit flow
 - Threat mitigations
 - Performance notes
 - Failure scenarios
-

Acceptance Criteria

- No legacy authentication implementation remains.
- All authentication endpoints use the production security services from Task 015.
- Argon2id is used for every password operation.
- RS256 JWTs with key rotation are fully operational.
- Refresh token rotation and reuse detection work correctly.
- Sessions support multiple devices, revocation, idle timeout, and absolute timeout.
- Email verification, password reset, and MFA are fully functional.
- Audit logs are generated for every security-sensitive action.
- API behavior matches the OpenAPI contract from Task 006.
- The complete authentication flow is fully integrated across FastAPI → Application → Domain → Infrastructure → PostgreSQL with comprehensive integration tests.

Task 017 — Background Processing, Outbox Dispatcher & Notification Platform

Objective

Implement the complete asynchronous processing infrastructure described throughout Tasks 003–016.

Replace every placeholder background process with production-grade implementations.

This task completes the event-driven architecture by wiring Domain Events → Transactional Outbox → Background Workers → Subscribers.

No business logic may execute directly inside HTTP request handlers unless explicitly required for strong consistency.

Deliverable

mastery-engine/

```
backend/  
  app/  
    workers/  
    infrastructure/  
      events/  
      notifications/  
      email/  
      scheduler/  
      queue/  
      redis/
```

```
docs/  
  background-processing/  
  notifications/
```

```
tests/
```

Expected size:

- 5,000–8,000 lines
 - 30–45 new files
 - 300+ automated tests
-

Objectives

Complete production implementations for

- Transactional Outbox
 - Event Dispatcher
 - Background Worker Host
 - Notification Service
 - Email Service
 - Retry Engine
 - Dead Letter Queue
 - Scheduler
 - Redis-backed Job Queue
 - Background Metrics
-

Worker Host

Create a production worker process.

Responsibilities

- Poll outbox
- Dispatch events
- Execute scheduled jobs
- Process retries
- Process notifications
- Send emails
- Cleanup expired tokens
- Cleanup expired sessions
- Generate analytics

Worker must support

Graceful shutdown

Health endpoint

Heartbeat

Worker identity

Multiple workers

Horizontal scaling

Transactional Outbox

Replace placeholder dispatcher.

Requirements

Exactly the architecture documented in ADR-0012.

Features

Batch polling

Visibility timeout

Retry count

Dead lettering

Poison message detection

Ordering guarantees

Worker locking

Lease expiration

Replay support

Metrics

Event Dispatcher

Implement subscriber registry.

Support

UserRegistered
UserLoggedIn

UserLoggedOut
AttemptRecorded
MasteryUpdated
WeakConceptDetected
ReviewScheduled
RecommendationGenerated
AchievementUnlocked
NotificationRequested
EmailVerificationRequested
PasswordResetRequested
SubscriptionChanged
OrganizationCreated
ContentPublished
AlgorithmPublished
SecurityIncidentDetected

Subscriber failures must never lose events.

Retry Engine

Implement exponential backoff.

Requirements

Retry schedule

1 minute

5 minutes

15 minutes

1 hour

6 hours

24 hours

Maximum retries configurable.

Permanent failures move to Dead Letter Queue.

Dead Letter Queue

Create

`dead_letter_events`

Track

Original event

Error

Stack trace

Retry history

Worker

Timestamp

Replay endpoint

Admin tools

Notification Service

Implement

Notification aggregate integration.

Support

In-App

Email

Future SMS abstraction

Notification preferences

Digest notifications

Priority

Expiration

Deduplication

Localization

Email Service

Implement

SMTP abstraction.

Templates

Verification email

Password reset

Welcome

Session alert

Security incident

Subscription

Achievement

Reminder

Weekly progress

System notification

Support

HTML

Plain text

Multipart

Attachments

Rate limiting

Retry

Bounce detection abstraction

Notification Preferences

Implement

Per user

Email enabled

In-app enabled

Digest frequency

Security notifications mandatory

Achievement notifications

Marketing notifications

Reminder notifications

Quiet hours

Timezone awareness

Scheduler

Implement background scheduler.

Jobs

Review reminders

Queue generation

Cleanup expired sessions

Cleanup expired tokens

Cleanup refresh tokens

Daily analytics

Weekly summaries

Monthly reports

Backup verification

Heartbeat

Redis Queue

Implement

Reliable queue

Worker leasing

Visibility timeout

Delayed jobs

Priority queue

Scheduled jobs

Queue metrics

Background Metrics

Track

Queue depth

Dispatch latency

Worker utilization

Retry count

Dead letters

Email delivery

Notification latency

Processing throughput

Worker heartbeat

API

Create admin endpoints

GET /admin/workers

GET /admin/outbox

GET /admin/outbox/{id}

POST /admin/outbox/{id}/replay

GET /admin/dead-letters

POST /admin/dead-letters/{id}/retry

GET /admin/notifications

GET /admin/workers/metrics

GET /admin/jobs

POST /admin/jobs/run

Infrastructure

Implement

Worker Host

Dispatcher

Subscriber Registry

Redis Queue

SMTP Client

Notification Repository

Dead Letter Repository

Scheduler

Metrics Collector

Heartbeat Service

Testing

Implement

Worker lifecycle

Outbox dispatch

Retry logic

Dead lettering

Subscriber failures

Replay

Notification delivery

Email rendering

SMTP failures

Scheduler

Redis queue

Worker concurrency

Ordering

Graceful shutdown

Duplicate prevention

Load testing

Target

300+ tests

Documentation

Create

`docs/background-processing/`

`README.md`

`worker-architecture.md`

`outbox-pattern.md`

`dispatcher.md`

`scheduler.md`

`notifications.md`

`email.md`

`dead-letter-queue.md`

`retry-policy.md`

`operations.md`

Acceptance Criteria

- Transactional Outbox is fully operational.

- Background workers process events independently of HTTP requests.
- Retry engine with exponential backoff is implemented.
- Dead Letter Queue captures unrecoverable failures.
- Notification system supports in-app and email delivery.
- SMTP abstraction with production-ready templates is implemented.
- Scheduler executes recurring jobs reliably.
- Redis-backed queue supports delayed and priority jobs.
- Background metrics expose worker health and throughput.
- Replay of failed events is supported.
- No domain logic executes inside worker infrastructure; workers orchestrate application services only.
- All implementations align with ADR-0012, Tasks 005, 010, and 012.
- Comprehensive automated tests validate reliability, concurrency, ordering, and failure recovery.

Task 018 — Frontend Foundation & Authentication Platform

Objective

Build the production frontend foundation for the Mastery Engine using the existing backend from Tasks 001–017. This task establishes the design system, routing, authentication, API integration, layouts, and application shell. No mock data, placeholder APIs, or fake authentication are allowed.

Deliverable

mastery-engine/frontend/

A complete production-ready Next.js application integrated with the existing FastAPI backend.

Expected size:

- 90–120 frontend files
- ~10,000–15,000 lines
- 300+ frontend tests

Technology Requirements

Must use:

- Next.js 15 App Router
- React 19
- TypeScript (strict)

- Tailwind CSS
- React Query (TanStack Query)
- React Hook Form
- Zod
- Zustand (minimal global state)
- Framer Motion
- Axios (or existing API client)
- next-themes
- Recharts
- Playwright
- Vitest
- Testing Library

No Redux.

No mock backend.

No fake APIs.

Build these sections

1. Design System

Create reusable UI components.

Examples:

- Button
- Input
- PasswordInput
- Checkbox
- Radio
- Select
- Dialog
- Drawer
- Sheet
- Tabs
- Tooltip
- Badge
- Avatar
- Alert
- Card
- Skeleton

- Spinner
- Toast
- EmptyState
- ErrorState
- Progress
- Breadcrumb
- Pagination
- DataTable
- Modal

Every component must include:

- accessibility
 - keyboard support
 - loading state
 - disabled state
 - variants
 - dark mode
-

2. Layout System

Build:

Public Layout

Authenticated Layout

Admin Layout

Each includes:

- responsive sidebar
 - mobile navigation
 - header
 - breadcrumbs
 - profile menu
 - notification menu
 - loading boundary
 - error boundary
-

3. Theme System

Implement:

Light

Dark

System preference

Persist selection.

4. Authentication UI

Replace placeholder authentication.

Pages:

Login

Register

Forgot Password

Reset Password

Verify Email

MFA Setup

MFA Verification

Recovery Codes

Session Expired

Unauthorized

Forbidden

Features:

React Hook Form

Zod validation

password strength meter

show/hide password

remember me

loading states

field validation

API integration

JWT storage strategy matching backend

automatic refresh token handling

logout

logout all devices

5. API Integration Layer

Build:

typed API client

authentication interceptor

refresh token interceptor

error normalization

request retry

correlation ID propagation

idempotency key generation

file upload support

pagination helpers

query key factory

6. Route Protection

Implement:

Public routes

Authenticated routes

Admin routes

Permission-based rendering

Role-based rendering

Object-level authorization support

Unauthorized redirects

Session expiration handling

7. Global State

Only minimal global state:

Current user

Theme

Notifications

Sidebar state

Everything else must use React Query.

8. Forms

Reusable form system using:

React Hook Form

Zod

Server error mapping

Loading

Success

Optimistic validation

Reusable FormField components

9. Error Handling

Implement:

404 page

500 page

Offline page

Maintenance page

Network error UI

API error UI

Retry components

Error boundaries

10. Loading Experience

Skeleton loaders

Progress indicators

Optimistic transitions

Route loading UI

Suspense boundaries

11. Accessibility

WCAG AA

Keyboard navigation

ARIA labels

Focus management

Screen reader compatibility

Color contrast

Reduced motion support

12. Responsive Design

Desktop

Laptop

Tablet

Mobile

Collapsible sidebar

Touch-friendly controls

13. Testing

Vitest

Testing Library

Playwright

Minimum:

300 frontend tests

Component tests

Hook tests

Form tests

Authentication flow

Navigation

Accessibility

Dark mode

Responsive layout

14. Documentation

Create:

`docs/frontend/`

`README.md`

`design-system.md`

`routing.md`

`authentication.md`

`state-management.md`

`forms.md`

`testing.md`

`accessibility.md`

`theme.md`

`architecture.md`

Acceptance Criteria

- ✓ User can register
- ✓ User can login
- ✓ JWT refresh works automatically
- ✓ MFA works with backend
- ✓ Logout works
- ✓ Logout all devices works
- ✓ Theme switching works
- ✓ Responsive layouts work
- ✓ Dark mode works
- ✓ Components are reusable
- ✓ Forms use React Hook Form + Zod
- ✓ React Query integrated
- ✓ No mock data
- ✓ No placeholder authentication
- ✓ TypeScript strict passes
- ✓ ESLint passes
- ✓ Prettier passes
- ✓ Vitest passes
- ✓ Playwright passes
- ✓ Accessible UI
- ✓ Production-ready application shell

Constraints

- Do **not** modify Domain, Application, or Infrastructure business rules unless required for frontend integration.
- Consume the existing OpenAPI contract and backend endpoints from Tasks 006–017.
- Do **not** introduce duplicate business logic in the frontend; validation should complement, not replace, server-side validation.
- Prefer reusable components over page-specific implementations.
- Maintain compatibility with the existing Clean Architecture and bounded contexts.
- All frontend code must be production-ready, fully typed, and free of placeholder implementations.

Task 019 — Learner Portal & Complete Study Experience

Objective

Build the complete learner-facing application on top of the existing backend (Tasks 001–018). Replace the current frontend shell with a fully functional adaptive learning platform. Every screen must consume real APIs from the FastAPI backend. No mock data, fake questions, or placeholder dashboards are permitted.

Deliverable

mastery-engine/frontend/app/(learner) /

Approximately:

- 120–150 frontend files
- ~12,000–16,000 lines
- 400+ tests (Vitest + Playwright)

Build the complete learner experience

1. Dashboard

Implement a production learner dashboard showing live backend data.

Widgets:

- Welcome section
- Current streak
- Daily goal progress
- Queue remaining
- Due reviews
- Interview readiness
- Mastery overview
- Weak concepts
- Strong concepts
- Recent activity
- Weekly learning chart
- Monthly learning chart
- Recommendation card
- Continue studying
- Notifications
- Upcoming reviews

Support:

- loading
 - error
 - empty
 - offline
-

2. Subject Selection

Pages:

- Subject catalog
- Subject details
- Enrollment
- Continue learning
- Completed subjects

Features:

- search
- filtering
- progress indicators
- enrollment confirmation

Uses real Subject APIs.

3. Study Session Experience

Complete workflow:

Start session



Adaptive queue



Open question



Answer



Explanation



Recommendation



Next question



Session summary

Features:

- countdown timer
 - progress bar
 - pause/resume
 - abandon session
 - keyboard shortcuts
 - autosave
 - session recovery
-

4. Question Experience

Support every question type available.

Examples:

- Multiple choice
- Multiple select
- True/False
- Ordering
- Fill blank
- Matching
- Code output
- Short answer
- Numerical
- Future question types

Features:

- hints
- confidence slider
- timer
- accessibility
- markdown rendering
- syntax highlighting
- copy buttons

Never expose correct answers before submission.

5. Submission Flow

Consume the existing backend.

Display:

- correctness
- mastery delta
- explanation
- review schedule
- recommendation
- confidence analysis
- response time
- next action

Animations allowed but optional.

6. Session Summary

After session ends:

Questions answered

Accuracy

Time spent

Mastery gained

Weak concepts

Strong concepts

Recommendations

Review schedule

Achievements unlocked

Continue button

7. Mastery Pages

Create dedicated pages:

Overall mastery

Concept mastery

Concept history

Mastery timeline

Review history

Confidence trend

Evidence count

Weakest concepts

Strongest concepts

Use Recharts.

8. Review Center

Pages:

Due reviews

Upcoming reviews

Completed reviews

Calendar

Review statistics

Review history

Review details

Support:

today

week

month

9. Recommendations

Pages:

Current recommendations

Dismiss recommendation

Accept recommendation

Recommendation history

Recommendation details

Recommendation categories

Uses Recommendation APIs.

10. Achievements

Achievement gallery

Unlocked

Locked

Progress

Categories

Milestones

Celebration animations

Share dialog (optional)

11. Notifications

Notification center

Read/unread

Archive

Filter

Categories

Infinite scroll

Real API integration

12. User Profile

Pages:

Profile

Preferences

Learning settings

Notification settings

Privacy

Sessions

Security

MFA

Connected accounts

Delete account

13. Search

Global search

Subjects

Concepts

Questions (where authorized)

Recent searches

Search suggestions

Keyboard shortcut (Ctrl/Cmd + K)

14. Offline UX

Offline detection

Cached dashboard

Cached mastery

Queued mutations

Reconnect handling

Retry UI

Use React Query persistence where appropriate.

15. Performance

Route-level code splitting

Lazy loading

Image optimization

Virtualized long lists

Memoization

Prefetching

Bundle optimization

16. Accessibility

WCAG AA

Keyboard navigation

Screen readers

Focus management

Reduced motion

High contrast

Skip links

17. Testing

Minimum:

400 tests

Include:

- Dashboard
 - Session flow
 - Adaptive queue
 - Question rendering
 - Submission
 - Review pages
 - Mastery charts
 - Notifications
 - Profile
 - Offline behavior
 - Accessibility
 - Playwright E2E covering the full learning loop
-

18. Documentation

Create:

`docs/frontend/learner/`

`README.md`

`dashboard.md`

`study-session.md`

`question-flow.md`

`mastery.md`

`reviews.md`

recommendations.md

achievements.md

notifications.md

profile.md

offline.md

testing.md

Acceptance Criteria

- ☐ Learner can browse and enroll in subjects
 - ☐ Learner can start a study session
 - ☐ Adaptive queue loads from the real backend
 - ☐ Questions render correctly
 - ☐ Answers submit to the backend
 - ☐ Explanation displays after submission
 - ☐ Mastery updates immediately
 - ☐ Dashboard refreshes automatically
 - ☐ Review schedule updates
 - ☐ Recommendations display
 - ☐ Notifications work
 - ☐ Achievements display
 - ☐ Profile management works
 - ☐ Responsive on mobile/tablet/desktop
 - ☐ Accessible (WCAG AA)
 - ☐ TypeScript strict passes
 - ☐ ESLint passes
 - ☐ Playwright passes
 - ☐ 400+ frontend tests
-

Constraints

- Use only the APIs and contracts defined in Tasks 006–018.
- Do not duplicate backend business logic in the frontend.
- All learner-facing data must come from the backend.
- Maintain compatibility with the existing Clean Architecture and bounded contexts.
- Prefer reusable components and hooks over page-specific implementations.
- No mock data, placeholder content, or hardcoded business values.

Task 020 — Content Authoring Platform & Curriculum Management

Objective

Build the complete instructor/content-editor portal for creating, reviewing, versioning, publishing, and managing curriculum content. Integrate fully with the existing backend (Tasks 001–019), especially the Content System (Tasks 013–014). Every operation must use the real APIs and data model. No mock data or placeholder editors.

Deliverable

mastery-engine/frontend/app/(content) /

Expected size:

- 140–180 frontend files
 - ~18,000–24,000 lines
 - 500+ tests (Vitest + Playwright)
-

Build the complete Content Authoring Platform

1. Content Dashboard

Implement a dashboard for content editors showing:

- Subjects
- Draft templates
- Published templates
- Pending reviews
- Recently edited content
- Content packs
- Publishing queue
- Coverage statistics
- Concept graph summary
- Explanation coverage
- Misconception coverage
- Template quality metrics
- Version history
- Recent publishing activity

Support:

- loading
- error
- empty
- offline

2. Subject Management

Pages:

- Subject list
- Subject detail
- Create subject
- Edit subject
- Archive subject
- Publish subject
- Subject analytics

Features:

- search
- filtering
- status badges
- publish workflow
- draft/published indicators

Uses real Subject APIs.

3. Concept Management

Create complete CRUD for Concepts.

Support:

- Concept details
- Difficulty
- Importance
- Dependencies
- Prerequisites
- Related concepts
- Learning objectives
- Misconceptions
- Tags
- Slug editing

Visual dependency graph.

4. Learning Objectives

CRUD:

Create

Edit

Delete

Reorder

Bulk import

Bulk export

Markdown support.

5. Misconceptions

CRUD:

Common misconception

Severity

Remediation text

Examples

Related templates

Analytics

6. Question Template Builder

Build a full visual template editor.

Sections:

Prompt

Variables

Parameter schema

Correct answer generator

Distractor generator

Hint tiers

Explanation variants

Difficulty

Estimated duration

Concept mapping

Validation

Live preview

7. Live Question Preview

Real integration with QuestionFactory.

Features:

Generate preview

Choose seed

Replay

Different seeds

Variable inspection

Generated prompt

Generated answers

Distractors

Explanation preview

Render hash

Concept IDs

8. Explanation Editor

Support all explanation types:

Correct

Incorrect

Hint

Interview

Beginner

Markdown editor

Preview

Syntax highlighting

Variables

Version history

9. Template Versioning

Visual version history.

Compare versions.

Diff viewer.

Rollback.

Publish.

Archive.

Duplicate.

Restore.

10. Concept Mapping

Visual mapping UI.

Primary concepts

Secondary concepts

Future weight support

Dependency validation

Coverage visualization

11. Content Packs

Pages:

Create pack

Add templates

Remove templates

Version

Publish

Archive

Pack analytics

12. Publishing Workflow

Draft

↓

Review

↓

Approve

↓

Publish

↓

Archive

Features:

Approval dialogs

Publishing checklist

Validation

Dependency checks

Breaking-change warnings

13. Search

Global search across:

Subjects

Concepts

Templates

Learning objectives

Misconceptions

Content packs

14. Analytics

Charts for:

Template usage

Coverage

Difficulty distribution

Concept coverage

Question distribution

Publishing velocity

Content quality

Missing explanations

15. Bulk Operations

Bulk publish

Bulk archive

Bulk delete

Bulk export

Bulk tag

Bulk move

Bulk validation

16. Import / Export

Support:

JSON

Markdown

CSV

ZIP package

Preview before import.

Validation report.

17. Collaboration UX

Editor locking

Unsaved changes warning

Conflict detection

Version conflict resolution

Last editor

Recent activity

18. Accessibility

WCAG AA

Keyboard navigation

ARIA labels

Focus management

Screen reader support

Reduced motion

19. Performance

Lazy loading

Virtualized lists

Autosave

Debounced validation

Optimistic UI

Code splitting

20. Testing

Minimum:

500 tests

Include:

- Subject CRUD
 - Concept CRUD
 - Template Builder
 - Live Preview
 - QuestionFactory integration
 - Version history
 - Publishing workflow
 - Search
 - Bulk operations
 - Import/export
 - Accessibility
 - Playwright E2E covering the complete authoring workflow
-

21. Documentation

Create:

```
docs/frontend/content/
```

```
README.md
dashboard.md
subjects.md
concepts.md
template-builder.md
question-preview.md
versioning.md
publishing.md
analytics.md
bulk-operations.md
import-export.md
testing.md
```

Acceptance Criteria

- ☐ Content editors can create and manage subjects
 - ☐ Concepts, learning objectives, and misconceptions are fully editable
 - ☐ Question templates are created through a visual builder
 - ☐ Live QuestionFactory previews generate deterministic questions
 - ☐ Explanation variants are editable
 - ☐ Template version history and diffs work
 - ☐ Publishing workflow is operational
 - ☐ Content packs are manageable
 - ☐ Analytics display live backend data
 - ☐ Bulk operations function correctly
 - ☐ Import/export supports documented formats
 - ☐ Responsive on mobile/tablet/desktop
 - ☐ Accessible (WCAG AA)
 - ☐ TypeScript strict passes
 - ☐ ESLint passes
 - ☐ Playwright passes
 - ☐ 500+ frontend tests
-

Constraints

- Use only the APIs, domain model, and contracts established in Tasks 001–019.
- Do not duplicate QuestionFactory, TemplateEngine, VariableGenerator, or other domain logic in the frontend.
- All preview generation must call the backend and use the real deterministic generation pipeline.

- Maintain compatibility with the existing Clean Architecture and bounded contexts.
- Build reusable authoring components that can support future content types without architectural changes.
- No mock data, placeholder editors, or hardcoded curriculum content.

Task 021 — Administration Portal, Operations Console & Platform Management

Objective

Build the complete administration platform for the Mastery Engine. This portal is for administrators, organization admins, and system operators to manage the platform, monitor health, operate background workers, and perform administrative workflows. Integrate fully with the existing backend (Tasks 001–020). No mock data or placeholder dashboards.

Deliverable

mastery-engine/frontend/app/(admin) /

Expected size:

- 170–220 frontend files
- ~22,000–28,000 lines
- 600+ tests (Vitest + Playwright)

Build the complete Administration Platform

1. Operations Dashboard

Implement a live operational dashboard showing:

- Active users
- Daily active users
- Active study sessions
- Queue throughput
- Outbox backlog
- Worker status
- Dead-letter count
- Notification delivery
- Email delivery

- API latency
- Error rate
- Database health
- Redis health
- Background jobs
- Storage usage
- System version
- Feature flags enabled
- Active organizations

Support:

- loading
- error
- empty
- auto-refresh
- offline

2. User Management

Pages:

- User list
- User detail
- User profile
- User sessions
- MFA status
- Login history
- Activity timeline
- Suspend user
- Reactivate user
- Force password reset
- Force logout
- GDPR anonymization
- Delete account
- Audit history

Features:

- search
- filtering
- bulk actions
- role assignment
- permissions display

Uses real User APIs.

3. Organization Management

CRUD for organizations.

Support:

- Organization profile
 - Members
 - Roles
 - Invitations
 - Subject assignments
 - Usage statistics
 - Organization settings
 - Suspension
 - Archive
-

4. Role & Permission Management

Visual RBAC management.

Support:

- Roles
 - Effective permissions
 - Permission explorer
 - User role assignment
 - Organization roles
 - Admin overrides
 - Permission audit
-

5. Feature Flag Management

Pages:

- List flags
- Create
- Edit
- Delete
- Enable

- Disable
- Percentage rollout
- Organization targeting
- User targeting
- Schedule rollout
- Rollback

Display rollout status.

6. Notification Operations

Manage:

- Notification queue
- Delivery history
- Failures
- Retries
- User preferences
- Broadcast notification
- Test notification

7. Email Operations

Pages:

- Delivery log
- Failed deliveries
- Retry
- Bounce report
- Template preview
- Template testing
- Rate-limit status

8. Background Worker Console

Integrate Task 017.

Display:

Workers

Heartbeats

Leases

Current jobs

Queue depth

Processing rate

Retries

Failures

Worker logs

Restart worker

Drain worker

Pause worker

Resume worker

9. Outbox & Event Console

Pages:

Outbox events

Pending

Processing

Completed

Failed

Replay

Dead letters

Replay history

Poison messages

Lease inspection

10. Scheduler

Manage recurring jobs.

Display:

Upcoming jobs

Running jobs

Failed jobs

History

Execution duration

Manual execution

Pause

Resume

Disable

Enable

Cron visualization

11. Billing Administration

Manage:

Plans

Subscriptions

Invoices

Payments

Refunds

Organization subscriptions

Usage

Revenue dashboard

12. Audit Logs

Powerful audit viewer.

Support:

Filters

Users

Organizations

Time

Actions

Export

Correlation ID search

Timeline view

13. Security Center

Pages:

Security incidents

Failed logins

Suspicious sessions

MFA adoption

Token usage

Password resets

Permission changes

Rate limit violations

Threat dashboard

14. Platform Analytics

Charts for:

User growth

Learning activity

Content growth

Question usage

Mastery trends

Worker throughput

Email metrics

Notification metrics

Revenue

Organizations

System utilization

15. System Configuration

Editable configuration (where appropriate):

Feature toggles

Email settings

Queue settings

Scheduler settings

Limits

Timeouts

Notification defaults

Maintenance mode

16. Search

Global admin search across:

Users

Organizations

Subjects

Templates

Notifications

Jobs

Workers

Audit logs

Feature flags

17. Bulk Operations

Bulk:

Suspend users

Reactivate users

Delete notifications

Replay events

Retry emails

Assign roles

Feature rollout

Archive organizations

Export logs

18. Accessibility

WCAG AA

Keyboard navigation

ARIA

Focus management

Reduced motion

High contrast

19. Performance

Virtualized tables

Server-side pagination

Lazy loading

Route splitting

Optimistic UI

Prefetching

Memoization

20. Testing

Minimum:

600 tests

Include:

- User management
 - Organization management
 - RBAC
 - Feature flags
 - Workers
 - Scheduler
 - Outbox
 - Notifications
 - Email
 - Audit logs
 - Billing
 - Security dashboard
 - Analytics
 - Accessibility
 - Playwright E2E covering major admin workflows
-

21. Documentation

Create:

docs/frontend/admin/

README.md
operations-dashboard.md
users.md
organizations.md
rbac.md
feature-flags.md
workers.md
outbox.md

scheduler.md
notifications.md
email.md
audit.md
security.md
analytics.md
billing.md
testing.md

Acceptance Criteria

- ☒ Administrators can manage users and organizations
 - ☒ RBAC and permission management works
 - ☒ Feature flags can be rolled out and rolled back
 - ☒ Background workers are visible and controllable
 - ☒ Outbox events and dead letters can be inspected and replayed
 - ☒ Scheduler jobs can be managed
 - ☒ Notification and email operations are functional
 - ☒ Billing administration uses real backend APIs
 - ☒ Audit logs are searchable
 - ☒ Security dashboard displays live data
 - ☒ Analytics update from real backend metrics
 - ☒ Responsive on mobile/tablet/desktop
 - ☒ Accessible (WCAG AA)
 - ☒ TypeScript strict passes
 - ☒ ESLint passes
 - ☒ Playwright passes
 - ☒ 600+ tests
-

Constraints

- Use only the APIs and contracts established in Tasks 001–020.
- Do not duplicate backend business logic in the frontend.
- All operational data must come from the backend.
- Respect the existing RBAC model (learner, instructor, content_editor, organization_admin, administrator, system_admin).
- Build reusable administrative components (tables, filters, charts, inspectors) that can be shared across future modules.
- No mock data, placeholder metrics, or hardcoded operational values.

Task 022 — End-to-End Frontend ↔ Backend Integration, Real-Time Updates & Production Readiness

Deliverable:

A fully integrated Mastery Engine where every frontend screen communicates with the real backend, background workers operate continuously, real-time updates propagate through the UI, optimistic updates are implemented, production deployment is validated, and the application is ready for beta release.

Objective

Replace every remaining frontend stub, fake response, placeholder API, and mocked interaction with real backend integrations.

Every page built in Tasks 018–021 must become fully functional against the APIs from Tasks 011–017.

No demo mode.

No mock data.

No fake charts.

No placeholder notifications.

No local fake state.

Scope

Part 1 — API Integration Completion

Replace every remaining placeholder endpoint.

Every React Query hook must call the production backend.

Includes:

- Authentication
- Dashboard

- Study Sessions
- Queue
- Questions
- Attempts
- Reviews
- Recommendations
- Notifications
- Search
- Subjects
- Content
- Admin
- Billing
- Workers
- Scheduler
- Feature Flags
- Organizations
- Audit Logs
- Analytics

Acceptance

Every screen renders from PostgreSQL.

Part 2 — Real Authentication Integration

Replace development auth.

Implement production auth endpoints using

PasswordService

JWTService

SessionService

TokenService

MFAService

Features

Register

Login

Logout

Refresh

Forgot Password

Reset Password

Verify Email

MFA Setup

MFA Verify

Recovery Codes

Session Management

Device Management

Force Logout

Token Rotation

Part 3 — Cookie Authentication

Move authentication to

HttpOnly cookies

Refresh cookie

CSRF token

Automatic refresh

Silent renewal

Session expiration

Part 4 — WebSocket Gateway

Implement

/ws

Live updates for

Notifications

Dashboard

Worker metrics

Outbox backlog

Scheduler

Security incidents

Session expiration

Queue updates

Study progress

Achievements

Part 5 — Live Notifications

Notification Center updates automatically.

No refresh.

Notification badge updates.

Unread count updates.

Toast notifications.

Priority handling.

Urgent notifications bypass quiet mode.

Part 6 — Live Dashboard

Dashboard widgets refresh automatically.

Charts animate.

Study streak updates live.

Mastery changes immediately.

Queue remaining updates.

Review due count updates.

Recommendation changes.

Part 7 — Background Worker Integration

Connect frontend admin console to

Worker Host

Dispatcher

Scheduler

Dead Letters

Outbox

Replay

Metrics

Everything updates in real time.

Part 8 — Optimistic UI

Implement optimistic updates for

Notifications

Recommendations

Feature flags

Content publishing

Template editing

Enrollment

Study Session

Answer submission

Rollback on failure.

Part 9 — Offline Support

Support

Offline detection

Queued mutations

Retry when online

Background sync

Cached dashboard

Cached content

Cached questions

Reconnect automatically.

Part 10 — React Query Optimization

Configure

Infinite queries

Prefetch

Cache invalidation

Mutation rollback

Dependent queries

Background refresh

Window focus refresh

Stale time optimization

Part 11 — Error Recovery

Global retry policy.

Network recovery.

API timeout handling.

Graceful degradation.

Maintenance mode.

Expired token recovery.

Offline banners.

Part 12 — File Upload Pipeline

Implement uploads for

Content Packs

CSV

Markdown

ZIP

Images

Progress indicators.

Cancellation.

Validation.

Virus scan hook.

Part 13 — Production Search

Wire global search to PostgreSQL.

Debounce.

Highlight matches.

Keyboard navigation.

Recent searches.

Saved searches.

Part 14 — Analytics Dashboard

Replace placeholder metrics.

Real

DAU

MAU

Sessions

Revenue

Retention

Worker throughput

Notification delivery

Email delivery

Queue depth

API latency

Database metrics

Part 15 — Feature Flag Integration

Feature flags dynamically enable

Routes

Buttons

Admin pages

Experiments

No rebuild required.

Part 16 — Complete Billing Integration

Connect

Plans

Invoices

Subscriptions

Checkout

Renewal

Cancellation

History

Payment status

Part 17 — Complete Audit Trail

Every admin action

Every learner action

Every content action

Every login

Every security event

shows inside Audit Logs.

Correlation IDs link every operation.

Part 18 — Accessibility Verification

Run full WCAG audit.

Fix

Keyboard traps

ARIA

Focus

Screen reader announcements

Contrast

Motion reduction

Part 19 — Performance Optimization

Implement

Route splitting

Lazy loading

Bundle optimization

Image optimization

Font optimization

Suspense boundaries

Memoization

Virtualized tables

Part 20 — Production Deployment

Validate

Docker Compose

HTTPS

Reverse Proxy

Worker

Scheduler

Redis

PostgreSQL

Environment variables

Secrets

Health endpoints

Rolling restart

Zero-downtime deployment

Testing

Expand to **700+ automated tests**, including:

- Unit tests
 - Integration tests
 - End-to-end tests
 - WebSocket tests
 - Authentication tests
 - Offline synchronization tests
 - Accessibility tests
 - Performance smoke tests
 - Production deployment validation
-

Documentation

Produce approximately **15–20 documents**, including:

- Integration Guide
- WebSocket Architecture
- Offline Strategy
- Deployment Guide
- Production Checklist
- Monitoring Guide
- Performance Guide
- Error Recovery
- Feature Flags

- Analytics
 - Security Operations
 - Incident Response
 - Backup & Restore
 - Scaling Guide
 - Release Process
-

Acceptance Criteria

- Every frontend page uses real backend APIs.
- Zero mock data remains.
- Authentication uses production services (Argon2id, RS256, refresh rotation, MFA).
- Real-time updates work across learner, content, and admin portals.
- Background workers update dashboards live.
- Offline mode supports queued actions and synchronization.
- Production deployment succeeds via Docker Compose.
- Monitoring, health checks, logging, and metrics are operational.
- End-to-end workflows (register → learn → mastery → review → admin operations) execute successfully against the real backend.
- The platform is suitable for a closed beta deployment.

Task 023 — AI Intelligence Platform, Provider Abstraction & Learning Analytics

Deliverable

Build the complete AI Intelligence Platform for the Mastery Engine as an **optional augmentation layer**. The deterministic learning engine remains the authoritative source for scoring, mastery, scheduling, and recommendations. AI must never replace business rules—it may only enhance explanations, analytics, insights, and authoring assistance.

Target approximately:

- **60–80 source files**
 - **15,000–20,000 lines**
 - **800+ automated tests**
 - **15–20 documentation files**
-

Core Architecture Principle

The Rule Engine always owns:

- Correct answer validation
- Scoring
- Mastery computation
- Review scheduling
- Adaptive queue ordering
- Achievement logic

AI may only consume the outputs of those systems.

Part 1 — AI Provider Abstraction

Create a provider-independent AI layer.

```
AIPProvider (interface)
```

↓

```
OllamaProvider (default)
```

↓

```
OpenAIPProvider
```

↓

```
GeminiProvider
```

↓

```
AnthropicProvider
```

↓

```
MockProvider (testing)
```

All providers must implement identical interfaces.

No application code may depend on any vendor SDK.

Part 2 — Local AI Runtime (Default)

Implement production support for

Ollama

Default model:

Qwen (latest stable 7B/8B class model available)

Configuration:

```
OLLAMA_HOST  
OLLAMA_MODEL  
AI_TIMEOUT  
MAX_CONTEXT  
MAX_TOKENS  
TEMPERATURE
```

Support streaming responses.

Support automatic model availability checks.

Graceful degradation when AI is unavailable.

Part 3 — AI Gateway

Build an AI Gateway responsible for

request validation

prompt construction

provider selection

fallback routing

timeouts

retries

response validation

token accounting

cost accounting

rate limiting

Part 4 — Prompt Management

Implement a prompt repository.

Version every prompt.

Support:

Explanation Prompt

Weakness Summary Prompt

Teacher Insight Prompt

Content Review Prompt

Recommendation Prompt

Weekly Report Prompt

Interview Coach Prompt

Every prompt has

version

owner

created date

approval state

Part 5 — AI Explanations

After answer submission

AI receives

Question

Student answer

Correct answer

Misconception

Mastery

Difficulty

Learning objective

Generate

Beginner explanation

Interview explanation

Detailed explanation

Simple explanation

Analogy

Common mistakes

Study tips

Rule-based explanation remains fallback.

Part 6 — Human Review Workflow

AI explanations are never automatically published.

Workflow

Draft

↓

Content Editor Review



Approve



Published



Student Visible

Support editing before approval.

Store revision history.

Part 7 — Personalized Study Coach

Generate

daily plan

weekly plan

motivation

study tips

weakness summary

next objectives

time recommendations

based on learner history.

Part 8 — Predictive Learning Analytics

Predict

dropout probability

course completion probability

mastery forecast

interview readiness forecast

daily study consistency

future review load

Model outputs confidence scores.

Part 9 — Instructor Intelligence

Provide instructors with

class weaknesses

concept heatmaps

misconception trends

recommended curriculum improvements

question quality alerts

student clusters

engagement summaries

Part 10 — Content Intelligence

Analyze content quality.

Detect

duplicate templates

unclear explanations

weak distractors

unused misconceptions

difficulty imbalance

missing objectives

coverage gaps

Generate improvement suggestions.

Part 11 — AI Recommendation Layer

Rule Engine produces recommendation.

AI rewrites recommendation into natural language.

Example

Instead of

Review Concept A

Generate

You struggled with recursion today.
Review it for 10 minutes before attempting the next exercise.

AI never changes recommendation type.

Part 12 — Weekly Reports

Automatically generate

Student Report

Instructor Report

Organization Report

Include

progress

mastery

weaknesses

strengths

recommendations

forecast

streak

study time

Export

PDF

Markdown

HTML

Part 13 — AI Safety Layer

Implement

prompt injection protection

PII removal

output validation

toxicity detection

hallucination confidence

maximum response length

citation support where applicable

unsafe output rejection

Fallback to deterministic explanations if validation fails.

Part 14 — AI Audit Trail

Log

provider

model

prompt version

latency

tokens

cost

response id

approval status

reviewer

Every AI interaction must be traceable.

Part 15 — Model Version Management

Support multiple models.

Qwen

↓

Gemma

↓

Llama

↓

OpenAI

↓

Gemini

Enable A/B comparison.

Rollback support.

Part 16 — Offline Evaluation

Replay historical attempts.

Compare

AI recommendation

Rule recommendation

actual learner outcome

Generate evaluation reports.

AI is never deployed without offline validation.

Part 17 — Experiment Framework

Support

A/B testing

feature flags

percentage rollout

organization rollout

user rollout

automatic metrics collection

Part 18 — Analytics Dashboard

Create AI dashboards.

Metrics include

latency

availability

provider usage

cache hit rate

fallback rate

approval rate

hallucination rejection

cost

token usage

quality score

Part 19 — AI Administration Portal

New admin pages

Providers

Models

Prompt Library

Prompt Versions

Evaluation Runs

Experiments

Safety Incidents

Audit Logs

Usage Metrics

Costs

Part 20 — Testing

Expand to **800+ automated tests**.

Include

- Unit tests
 - Integration tests
 - AI provider tests
 - Prompt validation tests
 - Safety tests
 - Audit tests
 - Human review workflow tests
 - Offline evaluation tests
 - WebSocket update tests
 - Performance tests
-

Documentation

Produce approximately **20 documents**, including:

- AI Architecture
- Provider Interface
- Ollama Deployment
- Prompt Engineering Guide
- Human Review Workflow
- Safety Controls
- Offline Evaluation
- AI Audit Model
- Model Versioning
- Experiment Framework
- Learning Analytics
- Instructor Intelligence
- Content Intelligence
- Cost Management
- Operations Guide
- Troubleshooting
- Security Considerations
- Deployment Guide
- API Reference
- Future Roadmap

Acceptance Criteria

- AI is completely optional and can be disabled without affecting core learning functionality.
- The deterministic Rule Engine remains the single authoritative source for scoring, mastery, scheduling, and adaptive learning decisions.
- Ollama with a Qwen-family model is the default local AI provider, with pluggable support for additional providers.
- All AI outputs pass through safety validation and are fully auditable.
- Human review is required before AI-generated educational content is published.
- Offline evaluation and experimentation are available before enabling AI features in production.
- AI augments the platform with explanations, insights, analytics, and authoring assistance without introducing vendor lock-in or compromising the correctness of the Mastery Engine.

Your architecture currently looks like this:

001-006 → Architecture + ADRs + Database Design
007-010 → Backend Foundation
011-017 → Production Backend
018-022 → Production Frontend
023 → Optional AI Platform

Status:		
Backend		100%
Frontend		100%
AI Platform		100%

Now comes the phase where most startups spend the majority of their engineering time.

Task 024 should NOT be another feature.

Instead it should make everything **actually production-ready**.

Right now you have:

- backend
- frontend
- worker system
- authentication
- AI
- content authoring
- notifications
- analytics

Now these all need to behave like **one product**.

Task 024 should become

Platform Hardening, Performance Optimization & Production Deployment

This is where companies usually spend months.

It includes things like

Performance

- Redis caching
 - cache invalidation
 - query optimization
 - database indexes
 - pagination optimization
 - lazy loading
 - virtual scrolling
 - image optimization
-

Load Testing

- 100 users
 - 500 users
 - 1000 users
 - concurrent study sessions
 - concurrent submissions
 - websocket stress
-

Security Hardening

- penetration testing
 - CSP refinement
 - rate limit tuning
 - audit verification
 - dependency scanning
 - secret scanning
-

Observability

- distributed tracing
 - OpenTelemetry
 - Grafana dashboards
 - Prometheus metrics
 - Sentry integration
 - request tracing
-

CI/CD

- GitHub Actions
 - Docker images
 - production builds
 - staging deployment
 - rollback deployment
 - blue-green deployment
-

Kubernetes (optional)

If later you want SaaS scale.

Disaster Recovery

- automatic backups
 - restore verification
 - worker recovery
 - Redis recovery
 - Postgres recovery
-

Production Monitoring

- uptime
 - CPU
 - memory
 - queue depth
 - worker latency
 - AI latency
 - websocket connections
-

Documentation

- deployment guide
- operations manual
- incident response
- runbooks
- scaling guide

You are a senior Staff Software Engineer.

Our platform has completed Tasks 001-024 and is production-ready for a closed beta.

Your goal is NOT to add new features.

Your goal is to transform the application into a production-quality Closed Beta platform for approximately 20 invited users.

Follow these rules:

- Preserve Clean Architecture.
- Do not break existing APIs.
- Maintain backward compatibility.
- Keep all automated tests passing.
- Add new tests where appropriate.
- Update documentation.
- Never introduce mock implementations.
- Keep production security.

Implement the following capabilities.

1. Closed Beta Mode

Add an application configuration:

CLOSED_BETA_ENABLED=true

When enabled:

- Registration is disabled unless an invite token exists.
- Public endpoints remain accessible only where appropriate.
- Login continues to work.
- Existing users continue to work.

2. Invite-only Registration

Implement invitation support.

Database:

beta_invites

Columns:

id

email

invite_token

expires_at

used_at

created_by

notes

Registration requires:

- valid token
- matching email
- unused
- not expired

Mark invite as used after successful registration.

3. Admin Invite Management

Add endpoints:

POST /admin/beta/invites

GET /admin/beta/invites

DELETE /admin/beta/invites/{id}

POST /admin/beta/invites/resend

Only administrators may access.

4. Beta User Limits

Configuration:

MAX_BETA_USERS=20

Prevent additional registrations after the limit.

Return HTTP 403.

5. Feature Flags

Create flags:

learning_enabled

content_authoring_enabled

ai_enabled

notifications_enabled

analytics_enabled

admin_console_enabled

Each flag must be dynamically configurable.

6. Beta Banner

Frontend:

Display:

"Closed Beta"

version

environment

feedback button

7. Feedback System

Add:

Feedback button

Modal

Rating

Category

Screenshot upload

Comment

Backend stores feedback.

8. Error Reporting

Allow users to report issues.

Automatically include:

correlation id

browser

platform

route

timestamp

9. Beta Analytics

Track:

daily active users

session duration

completion rate

registration count

drop-off points

question completion

review completion

10. Welcome Flow

After first login:

welcome wizard

profile

learning goal

subject selection

tutorial

11. Email

Invitation email

Welcome email

Beta reminder email

12. Documentation

Create:

closed-beta.md

beta-operations.md

invite-system.md

Deliver production-quality code.

Explain architectural decisions.

Generate migrations.

Generate tests.

Update OpenAPI.

Do not remove existing functionality.

Task 026 — Closed Beta Operations, Product Validation & User Success Platform

Objective

The Mastery Engine is feature-complete.

Do NOT build new product functionality.

Do NOT redesign the architecture.

Do NOT modify the Domain Model.

Do NOT rewrite existing APIs.

Your mission is to transform the platform from "software that is built" into "software that is being validated by real users."

Assume Tasks 001–025 are complete.

Everything already exists:

- FastAPI backend
- Next.js frontend
- PostgreSQL
- Redis
- Docker Compose
- Monitoring
- AI Platform
- Closed Beta

- Deployment Remediation

Now build the complete operational layer for a Closed Beta with 20–100 real learners.

=====

PART 1 — Beta Operations Dashboard

=====

Create a dedicated Beta Operations dashboard.

Show:

- Total invited users
- Active beta users
- Daily Active Users
- Weekly Active Users
- Invite conversion rate
- Average session duration
- Study sessions completed
- Feedback received
- Bugs reported
- Crash reports
- NPS score
- User satisfaction
- Learning progress

- Retention

=====

PART 2 — Product Analytics

=====

Build production-grade analytics.

Track:

Registration Funnel

Invite Sent

↓

Invite Accepted

↓

Registration

↓

Email Verification

↓

Welcome Wizard



First Enrollment



First Study Session



First Completed Question



Day-1 Retention

Measure:

- Funnel conversion
- Drop-off
- Time between steps
- Completion %

=====

PART 3 — Learning Effectiveness

=====

Measure:

Mastery growth

Time to mastery

Weak concepts

Strong concepts

Review effectiveness

Question accuracy

Average confidence

Hint usage

Recommendation acceptance

Adaptive queue quality

Interview readiness trend

=====

PART 4 — User Feedback Platform

=====

Expand the feedback system.

Include:

Bug reports

Feature requests

UI suggestions

Learning difficulty

AI feedback

Content feedback

Instructor feedback

Votes

Priority

Status

Roadmap linkage

Duplicate detection

=====

PART 5 — User Success Center

=====

Track:

Inactive users

Users at risk

Incomplete onboarding

Users stuck in learning

No study in 7 days

Failed registration

Email verification pending

Recommendation ignored

Generate actionable recommendations.

=====

PART 6 — Instructor Analytics

=====

Create dashboards showing:

Content quality

Concept coverage

Question quality

Template usage

Difficulty balance

Poor performing concepts

Frequently missed questions

Misconceptions

Explanation usefulness

=====

PART 7 — Operational Monitoring

=====

Build operations pages:

Platform Health

Worker Health

Background Jobs

Queue Status

Email Delivery

Notification Delivery

Database Health

Redis Health

Storage Usage

API Latency

AI Usage

Cost Metrics

=====

PART 8 — Release Management

=====

Implement:

Release notes

Version timeline

Feature freeze mode

Canary rollout tracking

Rollback history

Deployment history

=====

PART 9 — Beta Reports

=====

Generate automatically:

Daily Report

Weekly Report

Monthly Report

Include:

Growth

Retention

Learning outcomes

Feedback summary

Top bugs

Top requests

System health

=====

PART 10 — Experiment Platform

=====

Expand existing Experiment Framework.

Support:

A/B testing

Feature rollout %

Recommendation comparison

Queue comparison

Explanation comparison

AI vs Rule Engine

Statistical significance

Winner selection

=====

PART 11 — Admin Portal

=====

Extend Administration Portal.

Add:

Beta Operations

Product Analytics

User Success

Learning Insights

Experiment Console

Release Management

Feedback Review

Bug Triage

=====

PART 12 — Documentation

=====

Create professional documentation.

docs/beta/

closed-beta-playbook.md

user-success.md

analytics-guide.md

experimentation.md

release-management.md

operations-handbook.md

support-playbook.md

product-validation.md

=====

Testing

=====

Add comprehensive tests.

Target:

300+ new tests

No mocks where real systems exist.

Maintain 100% backward compatibility.

=====

Rules

=====

Do NOT add random features.

Do NOT redesign architecture.

Do NOT replace deterministic learning.

Use existing bounded contexts.

Use existing Clean Architecture.

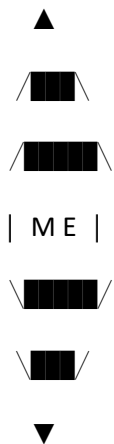
Use existing APIs.

Use existing RBAC.

Use existing AI provider abstraction.

Everything must integrate with Tasks 001–025.

The outcome should be a production-grade Closed Beta Operations Platform suitable for validating Mastery Engine with 20–100 real users before Public Beta.



MASTERY ENGINE

Closed Beta Operations

v1.0 Beta

Task 027 — Brand Identity, Public Website, Documentation Portal & Developer Ecosystem

Deliverable: Transform the Mastery Engine from an internal platform into a polished public product with a professional brand, landing website, documentation portal, SDKs, API explorer, onboarding, and developer ecosystem.

1. Brand Identity

Create a complete visual identity.

Brand Name

Instead of **Mastery Engine** as the public product name, consider:

☐ **MasteryOS** ☐ **(Recommended)**

Meaning:

The Operating System for Learning

Short

Professional

Easy to remember

Works globally

Perfect for SaaS

Examples:

MasteryOS

masteryos.com

docs.masteryos.com

api.masteryos.com

beta.masteryos.com

status.masteryos.com

Alternative names

MasteryHub
MasteryCloud
LearnOS
SkillFlow
CogniCore
NovaLearn
MentorOS
EduFlow
KnowledgeOS
MasterGrid

Logo

Simple.

Minimal.

Modern.

Example concept:



MASTERY OS

Meaning:

Every level builds upward.

Or



MasteryOS

Hexagon = knowledge graph

Or

M

built from connected nodes

representing concept relationships.

Color Palette

Primary

#2563EB

Blue

Secondary

#7C3AED

Purple

Accent

#14B8A6

Teal

Success

#10B981

Warning

#F59E0B

Danger

#EF4444

Typography

Heading

Inter

Body

Inter

Code

JetBrains Mono

2. Public Marketing Website

/

Features

Solutions

Pricing

Security

Documentation

About

Contact

Blog

Careers

Roadmap

Landing Page

Hero

Features

Animations

Product screenshots

Testimonials

Architecture

FAQ

Call to Action

Newsletter

Footer

3. Documentation Portal

`docs.masteryos.com`

Sections

Getting Started

Installation

Architecture

REST API

WebSocket API

Authentication

SDKs

CLI

Deployment

Scaling

Monitoring

Security

AI

Learning Engine

Content Authoring

Administration

Troubleshooting

FAQ

Search included.

4. Interactive API Explorer

Swagger UI

Redoc

OpenAPI download

Example requests

Try-it-now

Authentication support

Rate-limit examples

Error catalog

5. SDKs

Official SDKs

Python

JavaScript

TypeScript

Go

Java

C#

Each should support

Authentication

Retry

Pagination

Streaming

WebSockets

Uploads

Downloads

Errors

Typed models

6. CLI

masteryos login

masteryos deploy

masteryos users

masteryos content

masteryos analytics

masteryos workers

masteryos backups

masteryos health

masteryos version

7. Public Status Page

status.masteryos.com

Shows

API

Workers

Database

Redis

WebSocket

Notifications

Email

AI

Uptime

Incidents

Maintenance

8. Product Roadmap

Public roadmap

Voting

Feature requests

Completed

Planned

In Progress

9. Changelog

Version history

v1.0

v1.1

v1.2

Release notes

[Breaking changes](#)

[Migration guides](#)

10. Blog

[Engineering](#)

[AI](#)

[Learning Science](#)

[Security](#)

[Product Updates](#)

[Architecture](#)

[Performance](#)

11. Customer Portal

[Account](#)

[Billing](#)

[Invoices](#)

[API Keys](#)

[Sessions](#)

[Devices](#)

[Usage](#)

[Organizations](#)

[Invitations](#)

Support

12. Support Center

Knowledge Base

Ticket System

Contact Form

Community

System Status

FAQ

13. SEO

OpenGraph

Twitter Cards

Sitemap

robots.txt

Structured Data

Canonical URLs

Metadata

Rich snippets

14. Analytics

Privacy-friendly analytics

Conversion funnel

Documentation usage

Search analytics

API usage

SDK downloads

15. Assets

Generate

Logo (SVG)

Favicon

App icons

Social banners

Email templates

Brand guidelines

Presentation deck

Press kit

Technical Requirements

- Maintain existing Clean Architecture and modular structure.
- No breaking changes to Tasks 001–026.
- Use existing authentication, RBAC, feature flags, monitoring, and API infrastructure.
- Generate reusable React components and TypeScript types.
- All public pages should support dark/light themes, responsive layouts, SEO metadata, and WCAG AA accessibility.
- Documentation should be generated from OpenAPI where appropriate and remain versioned.

- Add comprehensive tests (300+ new tests) covering branding assets where applicable, public website, documentation navigation, SDK generation, CLI commands, status page, and customer portal.
 - Provide detailed documentation for branding, design system extensions, deployment, and developer onboarding.
-

Expected Outcome

After Task 027, the platform will no longer look like an internal engineering project—it will present itself as a complete commercial SaaS product. A visitor should be able to discover **MasteryOS**, understand its capabilities, explore the documentation, try the APIs, monitor service status, download SDKs, and begin using the platform through a polished, professional experience.